Computers & Industrial Engineering 218 (2026) 112135

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

Research on a worker skill level-aware pulsating assembly scheduling method for aircraft based on the GIN-PPO algorithm

Xiaoyu Wen [a,b], Hao Zhang [a,b], Wenchao Yang [a,b], Hao Li [a,b,*], Haoqi Wang [a,b], Hongwen Xing [c], Siren Liu [c]

[a] Henan Provincial Key Laboratory of Intelligent Manufacturing of High-End Equipment, Zhengzhou University of Light Industry, Zhengzhou 450000, China
[b] School of Mechanical and Electrical Engineering, Zhengzhou University of Light Industry, Zhengzhou 450000, China
[c] COMAC Shanghai Aircraft Manufacturing Co., Ltd, Shanghai 200000, China

ARTICLE INFO

Keywords:
Graph isomorphism network
PPO algorithm
Aircraft assembly
Skill level

ABSTRACT

The cycle time of pulsating aircraft assembly accounts for nearly half of the entire aircraft manufacturing period. Its scheduling process faces challenges such as takt time synchronization of the pulsating line, multi-station collaboration, limited resources, and the complex coupling between worker skill levels and operational efficiency. However, existing studies rarely incorporate explicit quantitative modeling of worker skill-level heterogeneity into pulsating assembly scheduling, and most traditional approaches have limited capability in capturing complex structural job dependencies under large-scale, multi-constraint environments. To address these challenges, this paper proposes a pulsating aircraft assembly scheduling method based on Graph Isomorphism Network (GIN) and Proximal Policy Optimization (PPO) algorithms. First, an assembly scheduling model incorporating skill-level allocation constraints is constructed. The jobs and their dependencies are abstracted into a graph structure, and GIN is employed to extract deep features of job nodes, enabling accurate capture of complex job dependencies and resource demands. Second, within a reinforcement learning framework, the PPO algorithm is used for end-to-end training. By optimizing the scheduling policy, the method improves resource allocation efficiency and shortens the total completion time, while effectively handling complex resource limitations and skill-level constraints. Finally, multiple simulation experiments are conducted to validate the effectiveness of the proposed method, with comparisons made against traditional scheduling methods and modern optimization algorithms. Experimental results show that the proposed GIN-PPO approach significantly outperforms the comparison methods in terms of computational efficiency, solution quality, and convergence speed. It effectively reduces the total completion time of pulsating aircraft assembly jobs considering skill-level allocation, and demonstrates strong robustness and stability across test cases of different scales.

1. Introduction

With the increasing diversification and customization of aerospace products, aircraft assembly processes have become increasingly complex, especially under pulsating assembly line operations. In a pulsating aircraft assembly workshop, positioning devices, material handling equipment, assembly workers, buffers, and workstations must operate in a coordinated manner, while jobs flow across multiple stations under a fixed takt time. In such an environment, aircraft assembly scheduling directly affects production efficiency, resource utilization, and delivery lead time. During scheduling, operation precedence, worker efficiency, and labor-resource constraints must be considered simultaneously (Morandini et al., 2025).

Compared with conventional manufacturing systems, pulsating aircraft assembly exhibits several distinctive characteristics. Aircraft assembly is a discrete and labor-intensive process with large workloads, complex inter-job relationships, and strong dependence on manual operations (Wen et al., 2025). Therefore, scheduling decisions must not only determine job sequences, but also coordinate inter-station transfers, sectional space capacity limitations, multi-specialty collaboration, and the matching between worker skills and job requirements. The fixed takt-time rhythm and multi-station collaboration requirements further intensify the coupling among jobs, resources, and workspace conditions.

From a problem-classification perspective, aircraft assembly

* Corresponding author at: Zhengzhou University of Light Industry, No. 5 Dongfeng Road, Jinshui District, Zhengzhou, Henan, China.
E-mail address: lihao@zzuli.edu.cn (H. Li).

https://doi.org/10.1016/j.cie.2026.112135
Received 15 October 2025; Received in revised form 13 April 2026; Accepted 18 May 2026
Available online 21 May 2026
0360-8352/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

scheduling is closely related to the resource-constrained project scheduling problem (RCPSP). RCPSP was introduced to describe project scheduling problems under multiple resource constraints (Brucker et al., 2000), and it has long been recognized as an important NP-hard problem (Blazewicz et al., 1991; Pritsker et al., 1969). Over the past decades, many exact algorithms (Mingozzi et al., 1998; Sprecher et al., 2000) and heuristic approaches (Kolisch et al., 1999; Kolisch et al., 2006) have been developed, together with several practical extensions such as distributed multi-project scheduling (Li et al., 2018), expanded resource types (Alcaraz et al., 2003), and resource preemption (Deblaere et al., 2011). However, aircraft assembly scheduling cannot be fully captured by classical RCPSP formulations. In addition to precedence and resource constraints, aircraft assembly must explicitly address sectional space limitations, shared equipment usage, multi-trade coordination, and worker skill heterogeneity. More importantly, worker skill levels not only affect assignment feasibility, but also influence the effective processing duration of jobs, which makes the interaction between resource allocation and schedule optimization much more complex.

In aircraft manufacturing, the pulsating assembly line typically operates under a production paradigm characterized by fixed takt time, station collaboration, and dynamic transfer. Differences in job volume, spatial capacity, and resource allocation across stations, together with competition among concurrently executable jobs, substantially increase the complexity of completion-time estimation and schedule generation (Blanco et al., 2024). Traditional RCPSP models usually assume fixed job durations and predefined precedence-resource structures. By contrast, practical aircraft assembly scheduling involves heterogeneous workforce configurations, skill-dependent execution efficiency, and dynamic adjustments during production. These characteristics limit the applicability of conventional formulations and motivate the development of more expressive and scalable solution methods.

With the development of Industry 4.0, technologies such as digital twins, the Internet of Things, and cyber-physical systems have provided new possibilities for real-time perception and adaptive scheduling (Zheng et al., 2025; Nudtasomboon et al., 1997). In actual aircraft assembly environments, however, production is still frequently affected by uncertainties such as material shortages and worker absences (Zarei et al., 2023). Therefore, a generalized scheduling method is needed that can handle different job structures, heterogeneous resource configurations, and skill-matching requirements while maintaining both solution quality and computational efficiency. Overall, existing studies on aircraft assembly scheduling still face three major limitations. First, the quantitative effect of worker skill heterogeneity on scheduling performance has not been sufficiently modeled. Second, traditional heuristic and genetic-based methods have limited ability to represent complex structural job dependencies in large-scale, multi-constraint assembly environments. Third, many existing approaches still show limited scalability and adaptability when problem size and resource configurations vary. To address these issues, this paper proposes a worker skill level-aware pulsating aircraft assembly scheduling method based on a Graph Isomorphism Network (GIN) and Proximal Policy Optimization (PPO).

Specifically, this study first develops a scheduling model that jointly considers worker skill levels, resource allocation constraints, and sectional space limitations. Assembly jobs and their precedence relations are then represented as a graph structure, and GIN is employed to extract deep structural features so as to better capture complex job dependencies and resource requirements. Based on this representation, PPO is adopted to optimize the scheduling policy in an end-to-end manner, enabling stable and effective decision-making in large-scale, multi-constraint scheduling environments. The main contributions of this paper are threefold, each addressing existing limitations in current aircraft assembly scheduling research: (1) to overcome the insufficient modeling of worker skill-level heterogeneity in pulsating assembly environments, an aircraft assembly scheduling model is developed that simultaneously considers worker skill levels, spatial capacity, and multi-skilled collaboration, explicitly capturing the quantitative coupling between skill allocation and job duration; (2) to address the limited capability of traditional heuristic and genetic-based methods in representing complex structural job dependencies, a novel integration of GIN and PPO is proposed, enabling a deep fusion of structural modeling and policy optimization; and (3) to improve the scalability and generalization performance of intelligent scheduling methods under large-scale, multi-constraint scenarios, a training strategy based on a relatively small dataset combined with data augmentation and experience replay is designed, demonstrating stable robustness across different problem scales and industrial settings.

The remainder of this paper is organized as follows. Section 2 reviews the related literature. Section 3 presents the problem description, mathematical model, and Markov decision process formulation. Section 4 introduces the proposed GIN-PPO framework. Section 5 reports the experimental design and comparative results. Section 6 discusses the findings and limitations. Section 7 concludes the paper and outlines future research directions.

2. Literature Review

Aircraft assembly scheduling is closely related to the resource-constrained project scheduling problem (RCPSP), whose solution approaches have evolved from exact algorithms to heuristic, metaheuristic, and intelligent optimization methods over time (Bahroun et al., 2024). However, practical pulsating assembly environments involve additional complexities such as spatial limitations, multi-trade coordination, and worker skill heterogeneity. To avoid repetition with the Introduction, this section focuses on the research streams most relevant to the present study from two perspectives: research on aircraft assembly scheduling and research on dynamic scheduling under uncertain environments.

2.1. Research on aircraft assembly scheduling

Existing studies on aircraft assembly scheduling have mainly focused on sequencing optimization, resource allocation, spatial coordination, and assembly-line balancing under multiple practical constraints. Wen et al. (2024) established a sequencing-, resource-, and space-constrained model and improved a genetic algorithm with Q-learning to optimize aircraft assembly scheduling, thereby effectively shortening the project duration. Bao et al. (2023a) addressed the joint optimization of resource allocation and scheduling in aircraft final assembly lines by proposing a surrogate-assisted heuristic approach to reduce resource investment while maintaining production efficiency. Guo et al. (2024) transformed the multi-constrained human-robot collaboration problem into a flexible job shop scheduling problem and solved it using a hybrid of mixed-integer linear programming and constraint programming. Lovato et al. (2022) investigated disruption management and rescheduling in aircraft assembly lines through a constraint-based scheduling framework. Li et al. (2025) studied the static scheduling problem of aircraft horizontal tail assembly and proposed an improved bilevel genetic algorithm to enhance assembly balance and solution efficiency under sequencing and resource constraints. Tereshchuk et al. (2021) investigated multi-robot assembly scheduling and proposed a reinforcement learning-based iterative auction method to improve scheduling efficiency, while Ren et al. (2020) combined branch-and-bound with a genetic algorithm to minimize manufacturing time for aircraft assembly projects under resource constraints. Tereshchuk et al. (2019) further developed a robot scheduling method based on workpiece geometry and structural features to improve multi-robot assembly efficiency. Shan et al. (2017) treated aircraft assembly scheduling within the RCPSP framework and proposed an adaptive genetic algorithm to optimize time, spatial capacity, and operator utilization. Bao et al. (2023b) also investigated balancing and scheduling in aircraft final assembly lines by formulating a mixed-integer optimization model

integrating resource constraints and parallel task scheduling. Hu et al. (2024) studied flexible assembly job shop scheduling and proposed a hybrid heuristic integrating mathematical programming with multi-objective evolution to optimize makespan and resource utilization. Huang et al. (2022) introduced a two-phase evolutionary algorithm for multi-objective distributed assembly flow shop scheduling, balancing makespan and energy consumption. Fang et al. (2021) focused on worker allocation in rhythmic aircraft final assembly lines and proposed a multi-objective evolutionary algorithm to minimize takt time and workload imbalance. Wang et al. (2024) incorporated worker preferences into a dynamic worker allocation model and a human–machine collaborative optimization method to improve assembly efficiency under worker shortage scenarios. Yan et al. (2022) integrated mixed-integer linear programming with a double-layer Q-learning approach to optimize aircraft assembly scheduling together with flexible preventive maintenance. Bao et al. (2025) investigated ramp-up planning for aircraft assembly production by integrating job allocation, scheduling strategies, and learning effects. Borreguero et al. (2024) studied a multi-mode RCPSP for aircraft assembly lines and proposed an event-based mixed-integer linear programming model combined with large neighborhood search to improve resource balance. Zhang et al. (2024) proposed a transfer-learning-enhanced elite genetic heuristic algorithm to improve global search capability in multi-constrained aircraft assembly scheduling.

In addition to these aircraft-specific studies, some related research has addressed scheduling robustness and uncertainty in assembly-oriented environments. Hu et al. (2024) proposed a network-based two-stage robust scheduling strategy for aircraft assembly systems with uncertain duration, improving scheduling stability through key-task identification and reverse resource optimization. These studies have enriched the literature on aircraft assembly scheduling under complex industrial constraints. Nevertheless, most of them mainly focus on static or semi-static optimization settings. Although some studies consider worker allocation, multi-skilled collaboration, or learning effects, worker skill levels are still rarely modeled as an explicit quantitative factor that directly affects job processing duration. Moreover, as the job scale expands and the coupling among precedence, space, and resource constraints becomes stronger, traditional heuristic, meta-heuristic, and mathematical programming methods may face increasing challenges in simultaneously maintaining modeling depth, computational efficiency, and solution quality.

2.2. Research on dynamic scheduling under uncertain environments

In practical assembly systems, production is often affected by uncertain disturbances such as material shortages, worker absences, machine failures, and fluctuating processing times. Under such conditions, the original production plan may no longer remain feasible or efficient, making dynamic scheduling and reactive rescheduling essential for maintaining production continuity and system robustness. Existing studies generally adopt two main response strategies: local adjustment, in which only the operations directly affected by disturbances are modified to restore production quickly with minimal changes; and global rescheduling, in which a new schedule is regenerated when disturbances are widespread or when local repair is insufficient (Mihoubi et al., 2021).

In this direction, Zhang et al. (2025) proposed a knowledge-transfer-based co-evolutionary algorithm for dynamic large-scale scheduling, leveraging historical experience to improve solution efficiency under changing environments. Villafranca et al. (2025) addressed uncertainty in aircraft maintenance durations through a two-stage stochastic scheduling model combined with an adaptive iterative local search heuristic. Wang et al. (2025) introduced a multi-agent deep reinforcement learning approach for dynamic flexible assembly job shop scheduling with uncertain processing and transportation times, enabling rapid and robust decision optimization. Zhu et al. (2021) proposed a reactive scheduling method for disturbances in aircraft moving assembly lines, focusing on uncertainty in material delivery. Wang et al. (2021) developed a double Q-learning method for adaptive scheduling under uncertain assembly times. Urgo et al. (2018) investigated scheduling under uncertainty in aircraft manufacturing from the perspective of order allocation and sequencing, emphasizing the balance between efficiency and flexibility. Boudjelida (2019) studied robust joint production and maintenance scheduling under uncertainty. Yan et al., (2022) combined digital twins with double-layer Q-learning to optimize dynamic scheduling and preventive maintenance, and Qiu et al. (2024) proposed a multi-level action-coupled reinforcement learning method for online scheduling under disturbance conditions.

Although these methods have achieved promising results in responding to dynamic disturbances, their application to aircraft assembly scheduling remains limited and exhibits two major limitations: (1) limited consideration factors – most studies focus on single disturbance types such as material delays or man-hour fluctuations, without incorporating the multi-dimensional coupled constraints unique to aircraft assembly, including skill-level variability and spatial capacity constraints; (2) low computational efficiency in large-scale scenarios – as job volume increases, the computation time of existing global rescheduling methods remains high, restricting their practicality in real assembly operations.

In summary, existing research has mainly focused on improving genetic algorithms, ant colony optimization, or heuristic rules, with limited attention to worker skill adaptation and deep modeling of complex job dependencies. Table 1 illustrates the applicability and performance of different scheduling methods under various constraints and objectives. Compared with traditional heuristic and classical optimization algorithms, the GIN–PPO-based scheduling approach proposed in this paper handles complex job dependencies, resource allocation constraints, and dynamic skill adjustment more effectively. Through GIN-based graph modeling, the proposed method captures deep job dependencies and resource interactions, improving the accuracy and stability of scheduling decisions. Meanwhile, the PPO algorithm optimizes the scheduling policy end-to-end, achieving higher computational efficiency and solution quality under large-scale and complex constraints. Compared with other methods, the proposed approach demonstrates advantages in multi-skilled collaboration and skill matching, enhancing scheduling performance across different scales while addressing limitations in modeling depth and scalability.

3. Problem description and modeling

3.1. Problem description

The skill-level-aware pulsating aircraft assembly scheduling problem can be described as follows: scheduling the operations within one workstation of an aircraft assembly line. Among the J assembly operations in the same workstation, there are specific precedence constraints. Jobs with predecessor–successor relations must be performed in the specified order and cannot be skipped, whereas jobs without direct precedence relations may be executed in parallel when the corresponding resource, skill-level, and sectional space-capacity constraints are satisfied. Define the job set $J = [1, 2, \ldots . N]$ within the workstation, where Job 1 and Job N are virtual jobs representing the start and end of assembly, respectively. These virtual jobs do not occupy any resources, are not subject to operational constraints, and have a duration of 0.

In the assembly process within a workstation, each operation has corresponding resource requirements and skill-level requirements. For each trade-specific resource R_i, there is a fixed number n of workers, and each worker has a specific skill level. In this study, workers are classified into three skill levels: 1, 2, and 3. For a given job, when the total skill level of the assigned workers meets the skill-level requirement, the job can be completed within the normal duration; when the total skill level is lower than the requirement, the job duration is increased; conversely,

Table 1
Classification of references according to the characteristics of the scheduling problems.

References	Constraints and assumptions	Objectives	Solution method	Methods classification	Scale of the problem
Wen et al. (2024)	Task ordering, resource, and space constraints	Minimize assembly time	Q-learning improved genetic algorithm (GA)	Heuristic	Medium
Bao et al. (2023a)	Multi-objective scheduling, space constraints	Minimize resource utilization	Genetic algorithm combined with simulated annealing	Heuristic	Medium
Guo et al. (2024)	Human-robot collaboration, assembly line, resource constraints	Minimize makespan	Mixed-integer linear programming and constraint programming scheduling method	Hybrid	Medium
Lovato et al. (2022)	Efficient assembly line scheduling, workstation constraints	Minimize production time	Heuristic algorithms	Heuristic	Small
Li et al.(2025)	Multi-station assembly, worker skill constraints	Minimize makespan	Metaheuristic method	Metaheuristic	Large
Tereshchuk et al.(2019)	Multi-objective scheduling, space constraints	Minimize resource utilization	Genetic algorithm-based scheduling method	Heuristic	Medium
Shan et al. (2017)	Resource constraints, multiple scheduling objectives	Improve assembly time and resource utilization	Adaptive genetic algorithm	Heuristic	Medium
Hu et al. (2024)	Flexible assembly job shop, resource and time constraints	Minimize makespan	Hybrid heuristic scheduling method	Heuristic	Medium
Fang et al. (2021)	Rhythmic aircraft final assembly line, worker allocation, workload balancing	Minimize takt time and workload imbalance	Multi-objective evolutionary algorithm	Metaheuristic	Medium
Yan et al. (2022)	Aircraft assembly, task priority	Minimize task completion time	Task scheduling optimization method	Heuristic	Large
Borreguero et al. (2024)	Assembly line, dynamic task allocation	Improve resource utilization	Multi-objective optimization genetic algorithm	Heuristic	Medium
Zhang et al. (2025)	Dynamic large-scale scheduling, changing environments	Improve solution efficiency under dynamic environments	Knowledge-transfer-based co-evolutionary algorithm	Metaheuristic	Large
Huang et al. (2022)	Aircraft manufacturing, time constraints	Minimize total completion time	Mixed-integer programming	Exact	Medium
Wang et al. (2025)	Dynamic flexible assembly job shop, uncertain processing and transportation times	Enable rapid and robust scheduling optimization	Multi-agent deep reinforcement learning approach	Deep Reinforcement Learning	Large
Wang et al. (2021)	Multi-station assembly, resource allocation	Optimize resource utilization	Dual Q-learning-based adaptive scheduling method	Reinforcement Learning	Medium
Qiu et al.(2024)	Dynamic workstations, machine task scheduling	Minimize total completion time	Multi-level action-coupled reinforcement learning method	Reinforcement Learning	Large
This paper	Task dependencies, worker skill levels, resource constraints	Minimize total makespan, optimize resource and skill matching	Deep reinforcement learning scheduling method based on Graph Isomorphism Network (GIN) and Proximal Policy Optimization (PPO)	Deep Reinforcement Learning	Extra Large

when the total skill level is higher than the requirement, the duration is reduced. Under the constraints of trade-specific resources, sectional spatial capacity, and skill-level conditions, the objective of this problem is to reasonably schedule each operation so as to minimize the total

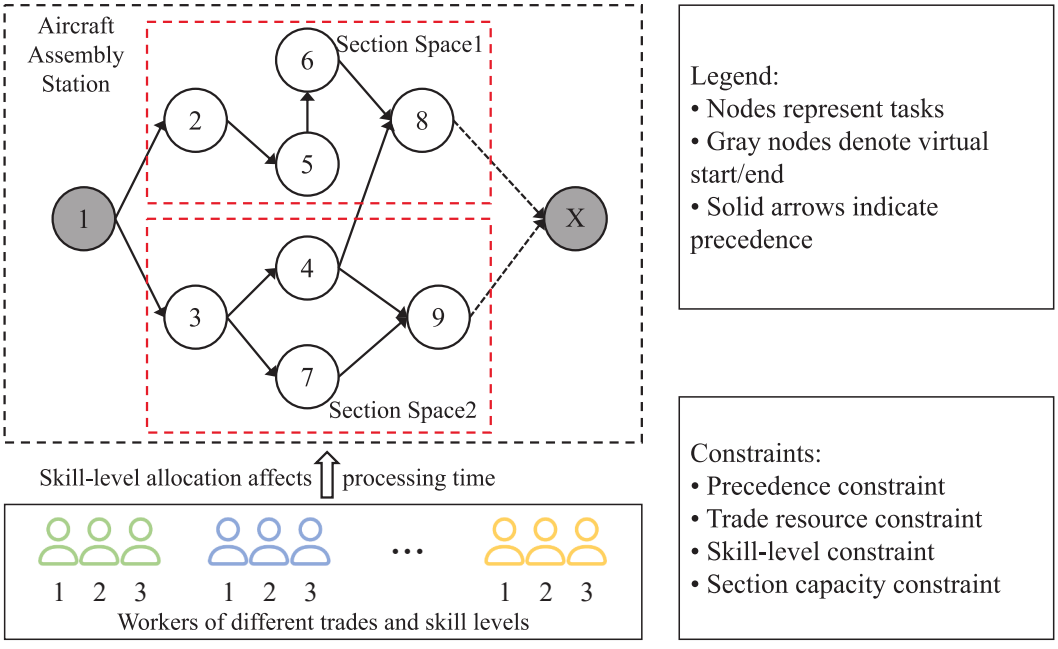


Fig. 1. Problem Characteristics Illustration.

completion time.

To enhance the clarity of the problem description, Fig. 1 presents a schematic illustration of the characteristics of the skill-level-aware pulsating aircraft assembly scheduling problem.

To better describe the aircraft assembly scheduling problem considering skill-level allocation, the symbols and their meanings are defined in Table 2. The assumptions of this scheduling problem are as follows:

1. During job execution, the types and amounts of resource requirements do not change, and the maximum supply of each resource does not change.

2. The spatial capacity differs across sections, and all assembly operations must satisfy the corresponding spatial constraints; operations assigned to different sections may be performed in parallel when the relevant precedence, resource, and skill-level constraints are satisfied.

3. The precedence relationships between jobs only consider the finish-to-start relationship, that is, the influence between the start of a job and the completion of its preceding job.

4. Before starting a job, all its predecessor jobs must be completed.

5. A job can start immediately after its previous job is completed, and once started, it cannot be interrupted.

3.2. Mathematical modeling of the problem

The mathematical model of the pulsating aircraft assembly line scheduling problem considering trade skill-level allocation is as follows:

Objective function: Minimize the total completion time of the pulsating aircraft assembly line scheduling.

$$MinZ = T \tag{1}$$

Decision variables:

$$\sum_{s\in S} x_{is} = 1 \forall i \in I \tag{2}$$

$$\sum_{k\in W_b} y_{ibk} = a_{ib} \quad \forall i \in I, \forall b \in B \tag{3}$$

Among them, decision variable (2) is the section space allocation variable. If job i is assigned to section space s, then $x_{is} = 1$; otherwise, $x_{is} = 0$. Variable (3) is the worker allocation variable, which indicates whether worker k in trade b is assigned to job i.

Table 2
Meaning of Related Symbols.

Symbol	Type	Meaning of Parameter
$J = \{1, 2, ..., N\}$	Set	Set of jobs, $i \in J$ denotes the i-th job
$B = \{1, 2, ..., b_{\max}\}$	Set	Set of trades,$b \in B$ denotes the b-th trade
$W_b = \{1, 2, ..., M_b\}$	Set	Set of workers in trade b, $k \in W_b$ denotes worker k
$S = \{1, 2, ..., s_{\max}\}$	Set	Set of sections, $s \in S$ denotes the s-th section space within the work station
P_i	Parameter	Basic working time of job i
T	Decision variable	Total completion time
N	Parameter	Total number of jobs
M_b	Parameter	Total number of resources in trade b
Δt_i	Parameter	Time adjustment amount of job i
a_{ib}	Parameter	Number of workers of trade b required by job i
m_{ib}	Parameter	Minimum required skill level in trade b for job i
α, β	Parameter	Adjustment coefficient for job i's working time when skill level is insufficient or exceeded
C_{bk}	Parameter	Skill level of worker k in trade b
G_s	Parameter	Maximum capacity of section space s
U_i^{bk}	Decision variable	Assignment of worker k in trade b to job i
S_i	Decision variable	Start time of job i
Z_{ib}	Decision variable	Binary variable indicating whether job i is assigned to trade b

Constraints:

$$S_i + P_i + \Delta t_i \le T \quad \forall i \in I \tag{4}$$

$$\sum_{s\in S} x_{is} = 1, \quad \forall i \in I \tag{5}$$

$$S_i + P_i + \Delta t_i \le Sj, \quad \forall (i,j) \in P \tag{6}$$

$$\sum_{i\in I}\sum_{k\in W_b} y_{ibk} \le M_b, \quad \forall b \in B \tag{7}$$

$$Z_{ib} = \sum_{k\in W_b} C_{bk} y_{ibk} \tag{8}$$

$$\Delta t_i = \sum_{b\in B} f_b(Z_{ib}, m_{ib}) \tag{9}$$

$$f_b(Z_{ib}, m_{ib}) = \begin{cases} \alpha(m_{ib} - Z_{ib}), & Z_{ib} \prec m_{ib} \\ -\beta(Z_{ib} - m_{ib}), & Z_{ib} \succ m_{ib} \\ 0, & Z_{ib} = m_{ib} \end{cases} \tag{10}$$

$$S_i + P_i + \Delta t_i \le S_j + M\left(1 - U_{ij}^{bk}\right) + M\left(2 - y_{ibk} - y_{jbk}\right) \tag{11}$$

$$S_j + P_j + \Delta t_j \le S_i + MU_{ij}^{bk} + M\left(2 - y_{ibk} - y_{jbk}\right) \tag{12}$$

$$\sum_{i\in I} a_{ib} \le M_b, \quad \forall t, \forall b \in B \tag{13}$$

$$\sum_{i\in I}\sum_{b\in B}\sum_{k\in W_b} y_{ibk} x_{is} \le G_s, \quad \forall s \in S, \forall t \tag{14}$$

Among them, constraint (4) indicates that all aircraft assembly line jobs must be completed within the total completion time and must not exceed the planned workstation takt time. Constraint (5) specifies that each job can only be assigned to one section space within the workstation. Constraint (6) represents the precedence constraint between related jobs, meaning that the start time of a successor job must not be earlier than the finish time of its predecessor job; it does not prohibit the parallel execution of unrelated jobs when the relevant constraints are satisfied. Constraint (7) states that, in each trade, the total number of assigned workers must not exceed the number of available workers in that trade. Constraints (8) and (9) enforce the minimum skill-level feasibility requirements for worker assignment. In contrast, Eq. (10) specifies the skill-level-dependent processing-time adjustment rule, which defines how the effective job duration is updated according to the degree of skill matching. Specifically, when the total skill level of assigned workers is lower than the minimum requirement, the job duration is increased; when it exceeds the minimum requirement, the job duration is reduced; and when it equals the minimum requirement, the job duration remains unchanged. Therefore, Eq. (10) represents a duration definition mechanism rather than a conventional feasibility constraint in mathematical programming. Constraints (11)–(12) specify that each worker k in the same trade b can only perform one job at any given time. If worker k is assigned to both job iii and job j, the two jobs cannot be executed simultaneously, and job iii must be completed either before or after job j. Constraint (13) represents the resource constraint, meaning that at any given time, the total resource demand for all simultaneously executed jobs in each trade must not exceed the total resource supply of that trade. Constraint (14) represents the spatial constraint, meaning that at any given time, the spatial capacity occupied by all workers assigned to jobs in that section must not exceed its maximum capacity. To complete the mathematical programming formulation, the domains of the decision variables are defined as follows:

$$x_{is} \in \{0,1\}, \forall i \in J, \forall s \in S$$
$$U_i^{bk} \in \{0,1\}, \forall i \in J, \forall b \in B, \forall k \in W_b$$
$$Z_{ib} \in \{0,1\}, \forall i \in J, \forall b \in B$$
$$S_i \geq 0, \forall i \in J$$
$$T \geq 0$$

3.3. Design of the Markov decision process

In this paper, the scheduling process of the pulsating aircraft assembly problem considering skill levels is formulated as a Discrete-Time Markov Decision Process (DT-MDP) under a serial scheduling generation scheme (SGS) (Olaguíbel et al., 1989). Here, the serial SGS is used as a schedule-construction mechanism: at each decision point, the agent selects one feasible job to be inserted into the schedule, while the actual execution timeline may still contain parallel operations whenever unrelated jobs satisfy the corresponding precedence, worker, resource, skill-level, and sectional space constraints. Through this dynamic process, the model progressively constructs a feasible scheduling solution to improve resource utilization and minimize makespan.

Fig. 2 shows the schedule-construction process under the Serial Schedule Generation Scheme (SGS). In this scheme, the policy selects one feasible job at each decision step and progressively inserts it into the schedule until all jobs have been assigned start times. This sequential construction procedure does not exclude parallel execution in the resulting schedule, because multiple unrelated jobs may overlap in time when all precedence, worker, resource, skill-level, and sectional space constraints are satisfied.

3.3.1. Stages and states

In the MDP model, a stage is defined based on the set of pending jobs X, i.e., the current set of schedulable jobs. When the predecessor jobs of a job have been completed and the required resources are available, the job is added to the pending job set and enters the current stage. If the pending job set is not empty, the system selects one job for schedule construction and proceeds to the next stage. This one-job selection mechanism is used only for incremental schedule generation; meanwhile, jobs that have already been started may continue in execution, and multiple unrelated jobs may overlap in time if all relevant constraints are satisfied.

In the MDP model, state S_i represents the state at stage i, which contains all the information required for decision-making, including the status of jobs, the status of resources, and the set of pending jobs, namely:

$$S_i = \left\{c_j, p_j, B_f, B_{kt}, X\right\} \tag{15}$$

where c_j represents the status of job j in the current stage: $c_j = 0$ indicates that job j has not started, or has been completed; $c_j = 1$ indicates that job j is in progress. P_j represents the remaining time required to complete job j. If job j has not started,$p_j = p_j^{init}$, i.e., the initial basic time of the job; if the job has been completed, then $p_j = 0$. B_j represents the amount of resources required by job j, i.e., the demand for each trade during job execution. B_f represents the remaining unallocated resources at the current time point. B_{kt} represents the current available capacity of each resource unit at the current time point, including the number of resources for each trade. The state of the initial stage S_0 is determined by the initial information of the scheduling problem, including the initial status of jobs, the initial resource capacity, and other relevant information, all of which are used as S_0.

3.3.2. Action design

In the MDP model, an action represents selecting one feasible job from the pending job set X for the current decision step of schedule construction. This action definition is adopted to sequentially build a feasible schedule under SGS; however, it does not imply that only one job can be in execution at a given time in the final schedule. The pending job set X contains the jobs that can be scheduled in the current stage, and a job is added to this set only when all its predecessor jobs have been completed and the corresponding worker availability, resource capacity, and skill-feasibility conditions are satisfied. Instead of selecting predefined priority rules, this paper adopts an end-to-end learning approach and directly takes activities in the aircraft assembly problem as scheduling actions. In this way, the action set dynamically changes during the solution process as the completion of predecessor activities allows new activities to become available for scheduling.

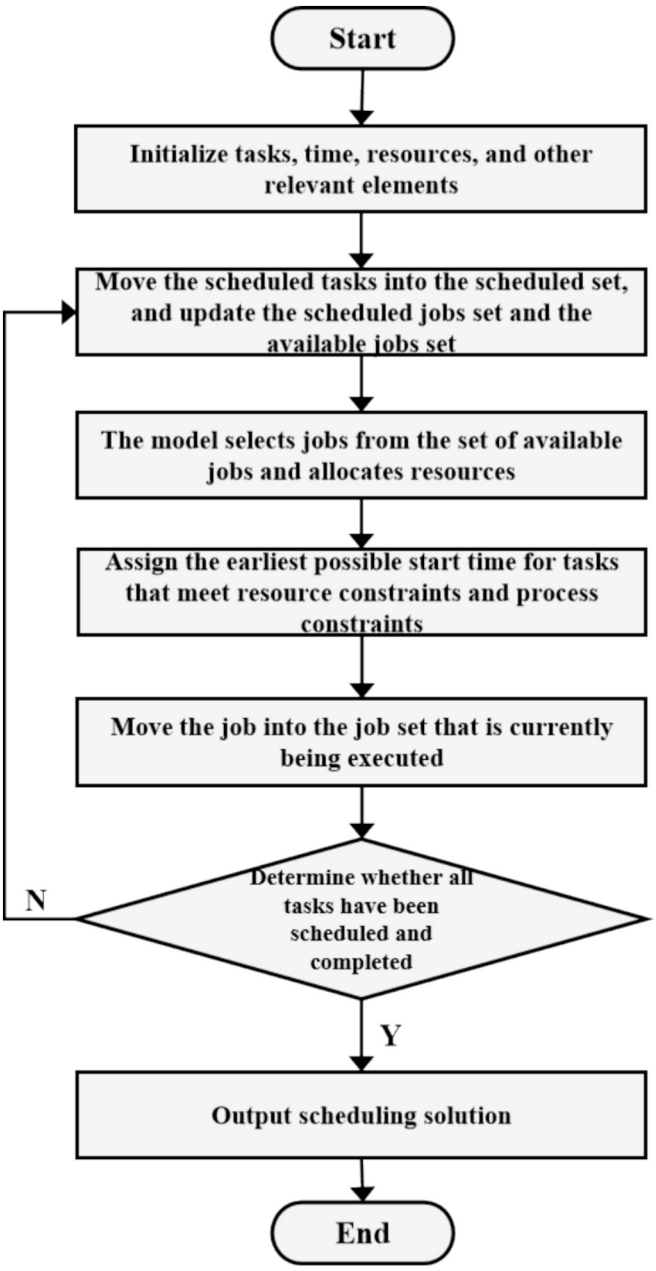


Fig. 2. Process flow of the Serial Schedule Generation Scheme (SGS).

3.3.3. Reward function design

In the pulsating aircraft assembly scheduling problem, the objective of reinforcement learning is to train the agent to learn a policy that optimizes job scheduling. The reward function serves as the key guiding signal in the learning process. In this study, a two-stage reward function is designed to balance multiple objectives, including minimizing the

total completion time, improving skill matching, and prioritizing critical path jobs.

1 First-stage reward: scoring based on total completion time

The first-stage reward is designed to minimize the total completion time, thereby improving scheduling efficiency and resource utilization by shortening the overall project duration. Since reinforcement learning seeks to maximize cumulative rewards, the first-stage reward is defined as the negative makespan:

$$R_1 = -makespan \tag{16}$$

2 Second-stage reward: reward design based on skill level and critical path

The second-stage reward further refines the scheduling process by considering two factors: skill matching and the prioritization of critical path jobs. Aircraft assembly scheduling depends not only on resource availability but also on the skill levels required for specific jobs. Appropriate skill matching can shorten job durations, improve efficiency, and avoid resource waste or delays; therefore, a skill-level reward is incorporated into the scheduling policy. In addition, the critical path represents the sequence of jobs that determines the total project completion time. Any delay in critical path jobs directly affects the makespan. Thus, a critical path reward is introduced to encourage the agent to prioritize jobs on the critical path, thereby reducing the overall makespan.

Skill matching plays an essential role in scheduling optimization. Since different jobs require different skill levels and workers' skills vary across trades, aligning allocated skills with job requirements improves execution efficiency. The agent adjusts job duration according to the allocated skill levels. We define a skill ratio r as the ratio between the required skill level of a job and the allocated skill level. A higher r indicates that the job's skill requirements are adequately satisfied, leading to a shorter duration, whereas a lower r implies insufficient skills and thus a longer duration. The skill ratio is calculated as follows:

$$r = \min\left(\frac{C_{bj}}{M_{bj}}\right) \tag{17}$$

Where: C_{bj} is the total allocated skill level of resource b for job j; M_{bj} is the minimum skill requirement of resource b for job j.

Based on the skill ratio r, the job duration is adjusted according to the degree of skill matching: better matching shortens the duration, whereas poorer matching extends it. The adjusted job duration T_a is calculated based on the skill ratio r as follows:

$$T_a = \begin{cases} T_b + \varphi(1-r), r \prec 0.95 \\ T_b - \varphi(r-1), r \succ 1.05 \\ T_b, 0.95 \le r \le 1.05 \end{cases} \tag{18}$$

The skill-level reward design is as follows:

$$R_2 = \begin{cases} -(T_a - T_b), r \prec 0.95 \\ T_b - T_a, r \succ 1.05 \\ 0, 0.95 \le r \le 1.05 \end{cases} \tag{19}$$

where: T_b is the basic duration of the job; T_a is the adjusted duration of the job;φ is the job time variation coefficient, and in this paper φ is set to 0.2.

Because delays on critical path jobs directly affect the total project completion time, an additional reward is assigned to encourage the prioritization of these jobs. The reward design for the critical path is as follows:

$$R_3 = \begin{cases} 1, \text{Critical Path} \\ 0, \text{other} \end{cases} \tag{20}$$

By combining the first-stage reward, the second-stage skill-level reward, and the critical path reward, the final comprehensive reward function is defined as:

$$R = \alpha R_1 + \beta R_2 + \chi R_3 \tag{21}$$

where α,β, and χ are coefficients used to adjust the importance of different reward components in the total reward. In this paper, their values are set as 1, 0.2, and 0.1, respectively.

Through this reward function design, the three objectives of minimizing total completion time, improving skill matching, and prioritizing critical jobs are jointly incorporated into the learning process, thereby providing the agent with a clear optimization direction. This design enables the agent to adaptively balance precedence, resource feasibility, and skill matching during policy learning, rather than treating worker skill constraints as static side conditions.

4. Algorithm design based on GIN-PPO

This section implements the Markov Decision Process (MDP) model constructed in Section 3.3 by designing an agent framework that combines a Graph Neural Network (GNN) with a reinforcement learning policy optimization mechanism. Specifically, this paper employs a GIN to model the topological structure between assembly jobs, extract high-order representations of scheduling states, and integrate a policy network based on the PPO algorithm to generate decision outputs. The mathematical programming model formulated in Section 3 has been fully implemented in the computational environment, and all constraints are explicitly embedded in the scheduling simulation. During training and inference, the proposed GIN–PPO agent generates scheduling decisions strictly within the feasible region defined by the mathematical model.

The proposed framework is problem-driven for skill-level-aware pulsating aircraft assembly scheduling. Since assembly jobs form a precedence-constrained network with strong structural dependencies, GIN is employed to encode the job graph and extract state embeddings. Meanwhile, because job durations are dynamically affected by worker skill-level allocation and the decision space is large and highly constrained, PPO is adopted to ensure stable learning and scalable convergence under multi-constraint scenarios.

The innovation and pertinence of the proposed framework can be summarized as follows. (1) Structural pertinence: GIN is adopted to learn high-order precedence and spatial dependency patterns from the assembly job graph, enabling structural generalization across different job networks. (2) Mechanism-level innovation: the quantitative coupling between worker skill levels and job processing durations is explicitly embedded into the environment dynamics and reward evaluation, linking resource assignment with scheduling performance in a unified learning loop. (3) Optimization pertinence: PPO provides stable policy updates for the large-scale, constrained combinatorial action space, allowing the model to generate feasible scheduling decisions without relying on manually designed priority rules or genetic operators.

The overall architecture of the proposed framework is shown in Fig. 3, which illustrates the interaction process from state input to action output and environment feedback.

4.1. Overall framework of the algorithm

In traditional scheduling optimization research, many methods rely on manually designed priority rules, and their performance heavily depends on the rationality and adaptability of these rules. To overcome this limitation, this paper proposes an end-to-end reinforcement learning method that directly generates scheduling actions through a policy network. Taking the scheduling state graph as input, the agent learns the policy function through training and achieves a direct mapping from state to action. Meanwhile, because aircraft assembly jobs exhibit significant structural precedence relationships and resource coupling constraints, this paper adopts a graph-structured modeling

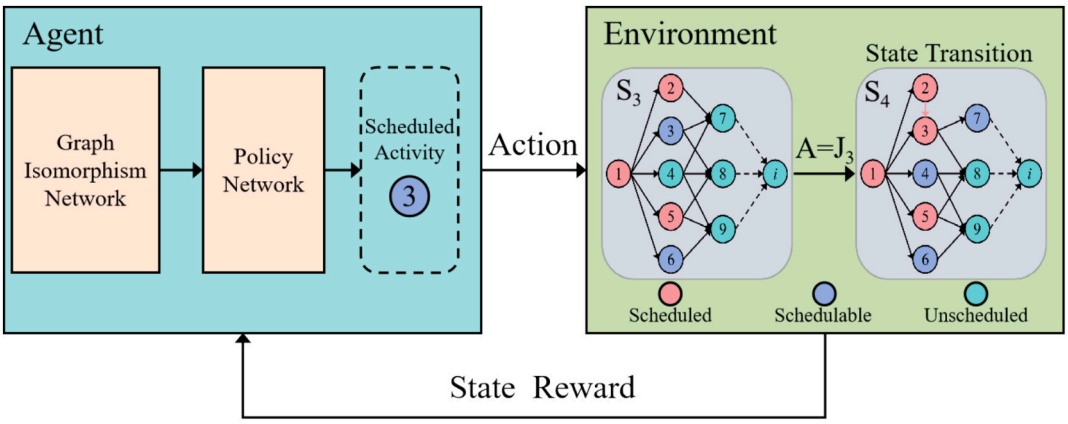


Fig. 3. Architecture of the Agent.

approach for scheduling states.

To specifically address the characteristic constraints of worker skill levels, the proposed GIN-PPO framework is adaptively designed at four levels. First, skill-related information, including trade-specific worker demand and required skill levels, is incorporated into the state and node-feature representation, enabling the agent to perceive skill heterogeneity during decision making. Second, skill-feasibility conditions are embedded into the environment and feasible-action generation mechanism, so that activities violating worker-skill requirements cannot be selected as valid scheduling actions. Third, the environment dynamically adjusts the effective processing duration of a job according to the degree of skill matching between assigned workers and job requirements, allowing the policy to learn the quantitative impact of skill allocation on schedule performance. Fourth, a dedicated skill-level reward is introduced to encourage the agent to prefer decisions with better skill matching, thereby improving both feasibility and execution efficiency. Through this integrated design, the proposed method can adapt its scheduling decisions to different worker-skill configurations rather than relying on fixed handcrafted dispatching rules.

In this work, the GIN is selected as the encoding model, which, on one hand, preserves the topological structure between jobs, and on the other hand, effectively extracts the contextual semantic features of each job, providing a representation basis for subsequent policy reasoning. Accordingly, this paper proposes an aircraft assembly scheduling optimization framework based on the GIN-PPO algorithm, as shown in Fig. 4:

The framework constructs the scheduling environment by extending the project network graph, encodes the state information within the scheduling environment using the GIN, and decodes it through an Actor-Critic model to designate project activities as output actions, while employing a masking mechanism to avoid selecting invalid activities. The agent continuously interacts with the scheduling environment, and based on data such as states, actions, and rewards, the PPO algorithm is used for offline training of the decision model, resulting in a trained scheduling policy model. This model can be directly applied to solve new scheduling instances, i.e., by obtaining the problem's state information from the scheduling environment as input to the optimal decision model, directly outputting the scheduled activities, and assigning resources and start times to them, ultimately yielding solutions to the skill-level-aware pulsating aircraft assembly scheduling problem.

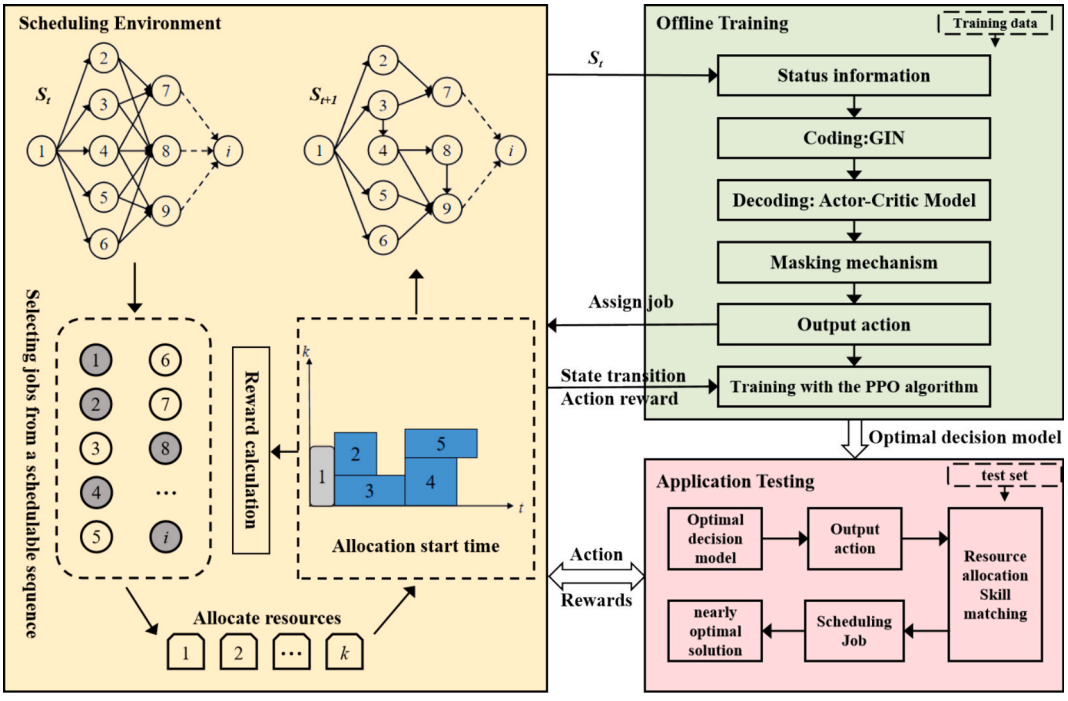


Fig. 4. Aircraft Assembly Scheduling Optimization Framework Based on the GIN-PPO Algorithm.

4.2. Agent construction based on GIN

GNNs are deep learning models designed for graph-structured data and can effectively capture dependencies between nodes through message passing. In the aircraft assembly scheduling problem, the dependencies between jobs can be modeled as a graph structure, where jobs are represented as nodes and precedence relationships are represented as edges.

4.2.1. Definition of graph structure and modeling method

In order to address the scheduling challenges in the pulsating aircraft assembly process — such as complex job sequences, limited resources, and skill-level constraints — this paper transforms the problem into a graph-structured representation and uses a GNN to generate high-dimensional feature embeddings of the scheduling state, which are then used as decision inputs for the reinforcement learning agent. The graph-structured modeling approach is able to preserve the topological information of jobs and incorporate node-level attributes, thereby comprehensively characterizing the job states and ensuring that the scheduling decisions better reflect complex constraint conditions. In this study, the GIN is adopted as the graph embedding model. GIN has the same discriminative capability as the Weisfeiler-Lehman graph isomorphism test and is well-suited for capturing the combined effect of complex dependency topologies and node states in scheduling graphs. Each GIN layer updates the node embeddings by aggregating the features of adjacent nodes, thereby effectively modeling the complex dependencies between jobs.

The aircraft assembly scheduling problem is constructed as a directed graph $G=(V,E)$, where $V=\{1,2,\ldots,N\}$ represents the set of assembly jobs, containing all aircraft assembly jobs to be scheduled;$E \subseteq V \times V$ represents the precedence constraints between jobs, i.e., if $(i,j) \in E$, it means that job j can only start after job iii is completed. Fig. 5 and Fig. 6 illustrate the project network diagram and a feasible solution for an aircraft assembly scheduling example, respectively. In Fig. 5, the black directed arcs represent the precedence constraints between jobs in the project without considering resource constraints; in Fig. 6, the red directed arcs indicate that originally parallel jobs have been rescheduled under the influence of resource constraints, resulting in a feasible scheduling scheme. It can be seen from the figures that resource constraints cause certain jobs that could originally proceed in parallel to be rescheduled, with additional constraint arcs introduced to reflect such adjustments.

Each job node $i \in V$ is associated with a set of structured features, representing its resource requirements, status, spatial attributes, and other related information. To enable the GIN to effectively process this graph, the initial feature vector of each node is defined as:

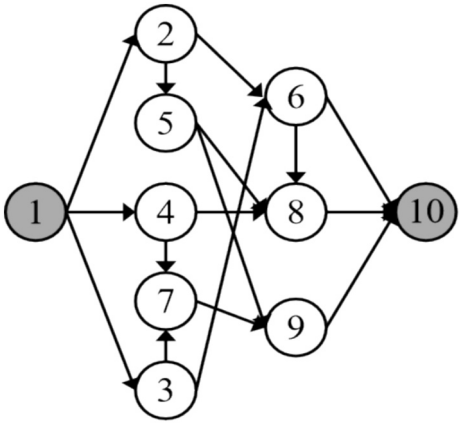


Fig. 5. Project Network Diagram of an Aircraft Assembly Example.

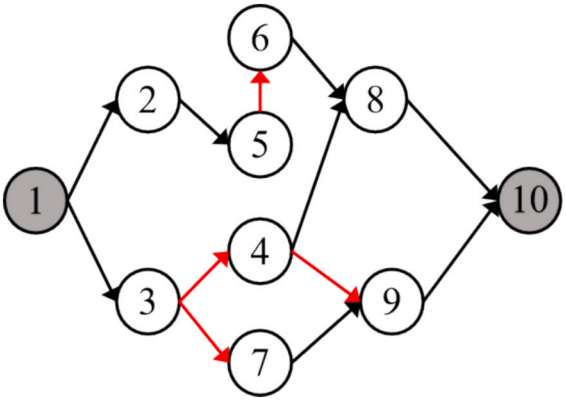


Fig. 6. Feasible Solution Representation of an Aircraft Assembly Example.

$$h_i = \left[p_i, q_i^{(1)}, q_i^{(2)}, \ldots, q_i^{(b)}, e_i, \sigma_i, S_i\right] \tag{22}$$

Where p_i is the basic working time of job i; $q_i^{(b)}$ is the resource demand of job i for trade b; $e_i \in \{0,1\}$ indicates whether the job has already been scheduled; $\sigma_i \in \{0,1\}$ indicates whether the job meets the schedulable condition, i.e., all predecessor jobs have been completed and the resource requirements are met; S_i denotes the section space number s where the job is located.

4.2.2. Policy network design

To better handle job dependencies, temporal features, and process logic features in aircraft assembly scheduling, this paper proposes a model architecture that combines a GNN with a Multi-Layer Perceptron (MLP). In this model, the state information of each job in the scheduling process is represented through the project network graph, while the temporal and process logic features of job nodes are computed through the graph structure. The interrelationships and dependencies between nodes are modeled using the GIN, in which each node's features (including temporal features and process logic features) are updated through message passing within the graph. In addition, the resource requirement features of jobs are encoded and processed by the MLP, further enhancing the model's representational capability in complex scheduling environments. The architecture of the network-based decision model is shown in Fig. 7. In this structure, the GIN captures complex relationships between job nodes through multiple layers of nonlinear transformations, producing a comprehensive vector representation of each job, which is then fed into subsequent MLP layers. The MLP primarily processes the encoding of resource features. By combining the two components, the model can simultaneously account for scheduling dependencies and resource requirements, thus enabling more accurate scheduling decisions.

In this decision model architecture, the scheduling decision process for jobs involves multiple encoding and decoding steps. The temporal features and process logic features of the jobs are encoded through the GIN. The GIN aggregates and updates the features of job nodes within the graph structure through multiple layers of nonlinear transformations. The resource requirement features of the jobs are encoded through the MLP. The MLP receives the resource demands of job nodes (such as the number of workers from different trades, required skill levels, etc.) and processes these features using a multi-layer perceptron. Through the MLP, the model can further capture the complexity of resource allocation and generate embedded representations of resource requirements. In this way, the policy network can explicitly distinguish jobs with different skill requirements and learn differentiated scheduling preferences under heterogeneous worker-skill configurations.

After processing with both the GIN and the MLP, the comprehensive features of each job node are combined into a unified representation

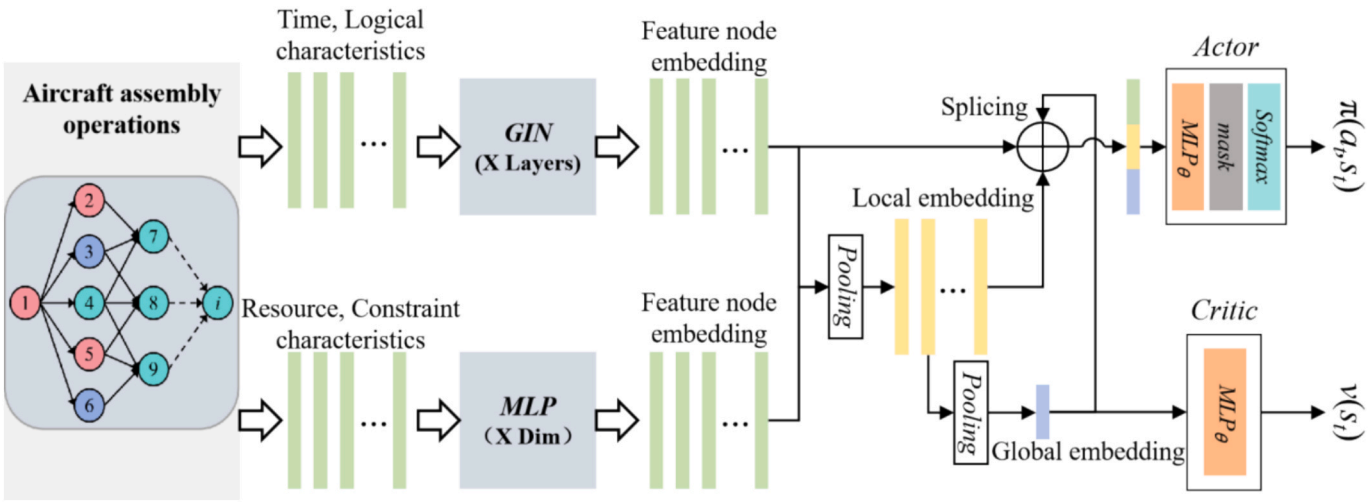


Fig. 7. Architecture of the Decision Model.

vector. This vector is then passed to the decoding stage. The decoding process normalizes the scheduling probabilities of jobs using the softmax function, thereby generating a scheduling score for each job. The softmax function converts the priority of each job into a probability distribution, ensuring that the agent can select the optimal job for scheduling based on the probability values of each job.

In particular, for temporal and logical feature encoding at decision step t, the state $S_t = G(V,E)$ and the feature vector of each node are encoded using the GIN. The GIN aggregates the features of neighboring nodes and iteratively updates its own node feature representations as follows:

$$h_v^{(l)} = MLP_{\theta_l}^{(l)}\left[(1+\varepsilon^{(l)})h_v^{(l-1)} + \sum_{u\in Z_{(v)}} h_u^{(l-1)}\right] l \in \{1,2,\ldots,L\} \tag{23}$$

where $h_v^{(l)}$ denotes the feature embedding of node v at the l-th layer;$h_v^{(0)} = [U_v \| W_v \| D_v]$ denotes the original feature of node v; $\|$ represents the concatenation operation;$MLP_{\theta_l}^{(l)}$ refers to the multi-layer perceptron; θ_l denotes the network parameter at the l-th layer; $\varepsilon^{(l)}$ represents the learnable parameters; $Z_{(v)}$ denotes the neighborhood set of node v. After L iterations of aggregation, a global representation of the entire graph can be obtained through the pooling function H, which takes as input the feature embeddings of all nodes and outputs the overall graph representation h_g^t, which is expressed as:

$$h_g^t = H\{h_v^{(L)} : v \in V\} = \frac{1}{|V|}\sum_{v\in V} h_v^{(L)} \tag{24}$$

Since the encoding of resource features is different from that of temporal and logical features, and resource features do not rely on the structural computations of the project network graph, an MLP is used to encode the resource feature E_v of each node v, specifically as follows:

$$b_v^{(d)} = MLP_{\mu_d}^{(d)}\left(b_v^{(d-1)}\right) v \in V; d \in \{1,2,\ldots,D\} \tag{25}$$

where $b_v^{(d)}$ denotes the resource feature embedding of node v at the d-th layer; $b_v^{(0)} = [E_v]$ denotes the original resource feature of node v; $MLP_{\mu_d}^{(d)}$ refers to the multi-layer perceptron;μ_d denotes the network parameter at the ddd-th layer. After D iterations of aggregation, a pooled vector representation b_g^t can be obtained through the pooling function F, where b_g^t is expressed as:

$$b_g^t = F\{h_v^{(D)} : v \in V\} = \frac{1}{|V|}\sum_{v\in V} h_v^{(D)} \tag{26}$$

In the decoding process, by performing decoding at each decision step t, a feasible solution for the aircraft assembly scheduling problem is constructed sequentially. The MLP is used to output the action coefficient c_v^t for each activity in the project, specifically as follows:

$$c_v^t = MLP_\omega\left(h_v^t \middle\| h_g^t \middle\| b_g^t\right) v \in V \tag{27}$$

By setting the action coefficients of already scheduled jobs and jobs that violate precedence constraints to negative infinity, the selection of illegal activities is masked, thereby preventing the generation of infeasible solutions. Then, the softmax function is applied to normalize the action coefficients of all activities, with the specific calculation given as follows:

$$\pi(a_t|S_t) = \frac{e^{c_v^t}}{\sum_{v\in V} c_v^t} \tag{28}$$

When training the model, actions are sampled according to the policy $\pi(a_t|S_t)$. Due to the stochastic nature of the sampling policy, the model is able to explore better scheduling solutions. For testing, a greedy strategy is adopted, where the trained policy $\pi(a_t|S_t)$ is used to greedily select the action with the highest probability.

4.3. PPO algorithm design

To better integrate the GIN network for processing graph-structured information among scheduling jobs, this paper adopts the PPO algorithm to optimize the decision policy. PPO is a policy-gradient-based reinforcement learning algorithm with high training stability (Meng et al., 2023). In this work, the input to the Actor–Critic networks consists of state feature embeddings computed by the GIN and MLP. The Actor network is used to learn the policy function, while the Critic network is used to estimate the value of the current state.

PPO constrains the magnitude of policy updates through a clipped surrogate objective, thereby improving training stability. Compared with value-based reinforcement learning methods, PPO is better suited to large and dynamically constrained action spaces such as those in aircraft assembly scheduling.

Specifically, the policy loss function, entropy loss function, and value loss function in PPO can be expressed as the following formulas:

$$L^{clip}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A\frown_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A\frown_t)] \tag{29}$$

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{30}$$

$$L_{entropy}(\theta) = -\beta\mathbb{E}_t[H(\pi_\theta(a_t|S_t))] \tag{31}$$

$$L_{value}(\theta) = \mathbb{E}_t\left[(v_\theta(S_t) - \widehat{R}_t)^2\right] \tag{32}$$

where $r_t(\theta)$ is the probability ratio between the new policy and the old policy (importance sampling ratio);$A\frown_t$ is the advantage function;ε is the clipping coefficient;β is a tuning coefficient used to control the impact of entropy on the total loss; H denotes entropy; $clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)$ is the clipping mechanism used to prevent large policy update steps. Through this clipping mechanism, PPO ensures during the optimization process that each update does not cause drastic changes to the policy, thereby improving training stability.$v_\theta(S_t)$ denotes the value of the state under the current policy;$\widehat{R}_t$ represents the reward obtained in the current state.

The total loss function of the PPO algorithm is calculated as follows:

$$L_{total}(\theta) = c_p L^{clip}(\theta) + c_e L_{entropy}(\theta) + c_v L_{value}(\theta) \tag{33}$$

where c_p is the policy loss coefficient; c_e is the entropy loss coefficient; and c_v is the value loss coefficient.

To make PPO algorithm training more efficient and stable, an integrated parallel training method is adopted. In each iteration, multiple instances are selected from the training set for parallel training, and the gradients of multiple agents are accumulated to update the network parameters. The detailed pseudocode is provided in Table 3.

5. Experimental design

In this study, a civil aircraft pulsating assembly line is adopted as the experimental benchmark. Based on Wen et al. (Wen et al., 2024), additional section-space attributes and worker skill-level requirements are incorporated to construct an enhanced dataset. The dataset includes five instance scales (30, 50, 100, 300, and 800 jobs) with 20, 20, 20, 10, and 10 instances, respectively, featuring varying network complexities. For scales 30, 50, and 100, 10 instances per scale are randomly selected for training, and the remaining instances are used for validation and testing. Hyperparameters are optimized via Bayesian optimization. Under the selected configuration, the model is trained on multi-scale training sets and evaluated on the corresponding test sets. All experiments are implemented in Python using PyTorch, and conducted on a workstation equipped with an RTX 1650 GPU, an Intel i7-12700F CPU (2.10 GHz), and 16 GB RAM.

Table 3
Pseudocode for PPO Training.

```
Begin
    Initialize training parameters
    for n = 1, 2, …,N do
    Select n AASP instances from the dataset
        for m = 1, 2, …, M do
            Initialize state S_t
        for t = 1, 2, …, T do
            Extract state features using GIN and MLP
            Sample action a_t from policy network π(·|S_t)
            Obtain reward R_t and next state S_{t+1}
            Update state: S_t ← S_{t+1}
            Record transition (S_t, a_t, R_t, S_{t+1})
      end for
        Compute Generalized Advantage Estimation (GAE) A⌢_t for each step
      Compute L^clip(θ), L_entropy(θ), L_value(θ)
      Optimize network parameters of the model within current step
      Update network parameters: θ_new ← θ_old + η(L_total(θ))
      end for
      If n % 100 ==0 then
            Validate policy π(·|S_t) on the validation set
        end if
    end for
End
```

5.1. Parameter tuning

To enhance the performance of the GIN-PPO model for the aircraft pulsating assembly scheduling problem, this paper adopts a hyperparameter tuning approach based on Bayesian optimization. Bayesian optimization is a global optimization method grounded in Bayesian theory, which uses a surrogate model to predict the objective function and guides the optimization process through historical data. This method can identify optimal solutions with relatively few evaluations. In this experiment, we implemented Bayesian optimization-based automated hyperparameter search using the Optuna library. The experiment set consisted of 10 job datasets from an aircraft pulsating assembly instance of scale 100, in which the first seven datasets were used as the training set and the last three as the validation set. Two evaluation metrics were adopted:

1. The average makespan across the three validation sets, which served as the primary evaluation metric;
2. The average training loss during the training process for each hyperparameter combination.

The hyperparameters to be tuned included: the number of MLP layers, the number of hidden units per MLP layer, PPO sampling steps per iteration, batch size, learning rate, and entropy coefficient. For each candidate combination, the system executed a complete policy training and evaluation process on all training instances, using the average makespan across all validation set instances as the performance measure for that combination. This result was then fed back to Optuna's Tree-structured Parzen Estimator (TPE) algorithm to suggest parameters for the next trial. The entire optimization process consisted of 300 trials, striking a balance between global search and local exploration. As shown in Fig. 8, the average makespan variation for all 300 hyperparameter combinations was recorded. It can be observed from the figure that the 211th trial yielded the optimal hyperparameter combination.

In the parameter optimization process shown in Fig. 8, the variation trend of the average makespan for each trial can be clearly observed. In the figure, the horizontal axis represents the number of trials, and the vertical axis represents the corresponding average makespan value. It can be seen that, during the early stages, the makespan values fluctuate significantly, indicating that the hyperparameter combinations selected in the initial phase did not effectively improve scheduling performance. However, as the optimization progresses, the fluctuation amplitude of the makespan gradually decreases, the optimization process becomes more stable, and the final optimization results begin to converge. This phenomenon verifies that the Bayesian optimization method is able to progressively guide the search process through its surrogate model, thereby identifying the optimal hyperparameter combination within a limited number of evaluations.

In this experiment, a boxplot (Fig. 9) is used to illustrate the impact of each hyperparameter on the makespan. In the figure, each box represents the distribution of makespan values under different hyperparameter settings, with gray and red colors used to distinguish results — where the red highlights indicate the optimal hyperparameter values. The detailed analysis for each hyperparameter is as follows:

1. Number of MLP layers: When set to 3, the median makespan is the lowest, and the fluctuation is minimal.
2. Number of hidden units: A value of 128 achieves a good balance between model expressiveness and training stability.
3. Sampling length: 2048 steps provide the best trade-off between advantage estimation accuracy and sample redundancy.
4. Batch size: A value of 32 ensures moderate gradient noise and a reasonable update frequency.
5. Learning rate (continuous parameter): The optimal value lies between 10^{-4} and 10^{-3}, ensuring an appropriate convergence speed.

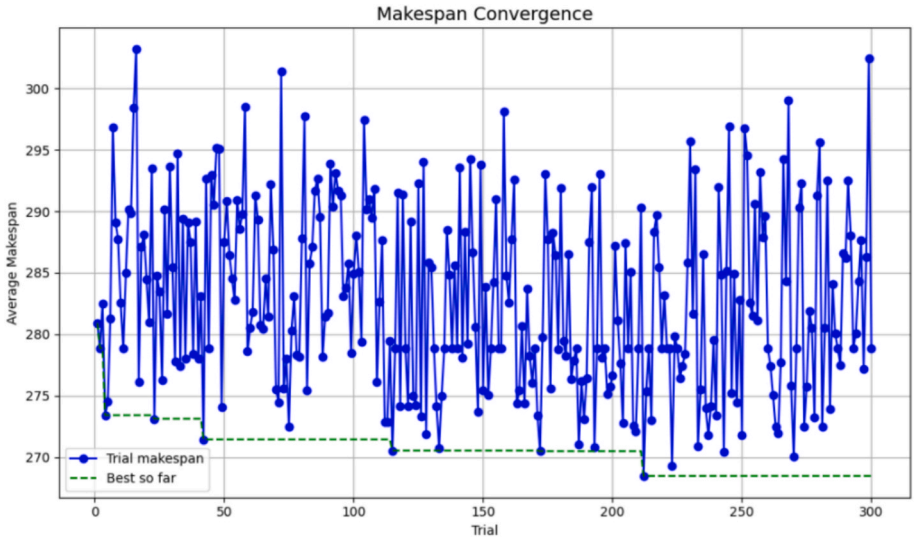


Fig. 8. Training curve for parameter optimization.

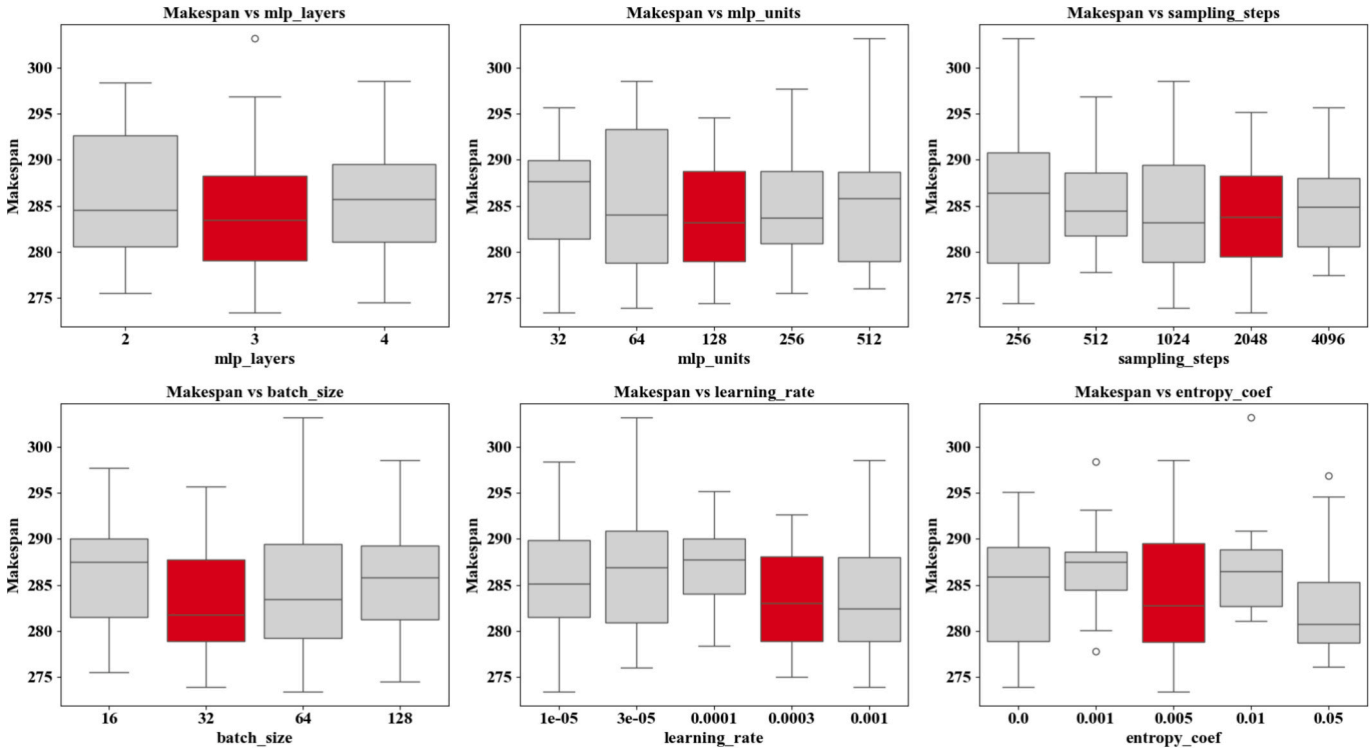


Fig. 9. Boxplots of Each Parameter.

6. Entropy coefficient (continuous parameter): Optimal values fall within the range 0.03 to 0.07, ensuring sufficient policy exploration capability.

Excessively high or low learning rates, as well as overly large or small entropy coefficients, can lead to unstable convergence or entrapment in local optima. Ultimately, after 300 trials, Optuna converged on the optimal configuration, which effectively reduced the makespan while maintaining low variability, demonstrating the effectiveness of Bayesian optimization in tuning complex scheduling jobs. The specific parameters adopted in this paper's algorithm are listed in Table 4.

Based on the experimental results in this section, it can be concluded that an appropriate choice of hyperparameters has a significant impact on the performance of the GIN-PPO algorithm. The Bayesian optimization–based tuning experiment effectively identified a

Table 4
GIN-PPO Hyperparameter Settings.

Hyperparameter	Value	Hyperparameter	Value
Number of MLP Layers	3	optimizer	Adam
Number of Neurons per Layer	128	clip_epsilon	0.2
Steps per Sampling Iteration	2048	gamma	0.99
Batch Size	32	max_episodes	2000
Learning Rate	0.0003	print_interval	50
Entropy Coefficient	0.005	hidden_channels	128

hyperparameter configuration that provides a good balance between training stability and scheduling performance.

5.2. Evaluation method

This study employs three metrics — makespan, runtime, and deviation rate from the optimal solution — to evaluate the performance of the algorithm. The deviation rate is calculated as shown in Equation (34), where the difference between the solution obtained by the proposed method and the optimal solution of the corresponding instance in the test set is used as the performance measure. The lower the deviation rate, the better the algorithm performance. Gap denotes the average deviation rate between the solution obtained by the algorithm and the optimal solution, which directly reflects the performance difference between the algorithm and the optimum.

$$Gap = \left(\frac{C_{\max,alb,p} - C_{\max,cp,p}}{C_{\max,cp,p}}\right) \times 100\% \tag{34}$$

Where $C_{\max,alb,p}$ is the project makespan obtained by the scheduling method for the aircraft pulsating assembly scheduling instance, and $C_{\max,cp,p}$ is the current best-known (optimal) makespan for the same instance.

5.3. Performance analysis of the proposed algorithm

In this section, the performance of the proposed GIN-PPO algorithm in the aircraft assembly scheduling problem is comprehensively evaluated and compared with several classical scheduling algorithms. To test the generalization capability of the model, the proposed approach is trained on 10 instances each of sizes 30, 50, and 100 to obtain a single best-trained model, which is then used to solve the test sets of sizes 30, 50, 100, 300, and 800.

Given the limited number of training instances, three strategies are employed to improve model performance: data augmentation, early stopping, and experience replay. Data augmentation increases the diversity of training samples, early stopping prevents overfitting, and experience replay enhances the utilization of historical training experiences. Through these strategies, the algorithm achieves stable training results even on relatively small training sets.

In addition to the final performance comparison, the convergence behavior of the proposed GIN-PPO scheduling framework is further analyzed over the entire training process. Fig. 10 illustrates the evaluation mean makespan together with its standard deviation throughout curriculum training. The horizontal axis represents the global training episodes, and the vertical dashed lines indicate transitions between different instance scales (30, 50, and 100 jobs).

It can be observed that within each stage, the evaluation mean makespan gradually decreases and stabilizes after sufficient training episodes, demonstrating consistent performance improvement. Although relatively larger variance appears in the early training phase, this is mainly caused by exploration during policy initialization. As training progresses, both the mean value and the variance tend to stabilize, indicating improved solution quality and increasing stability. The apparent performance shifts at stage transition points are due to changes in instance scale rather than instability of the optimization process. Overall, the results confirm that the proposed method exhibits stable convergence under progressive scale enlargement.

To further verify the convergence tendency within each curriculum stage, the evaluation mean makespan is normalized by the stage-wise best-so-far value, as shown in Fig. 11. For each stage, the normalized curve progressively approaches 1.0 as training proceeds, indicating that the algorithm converges toward stable high-quality solutions within that scale. This normalization eliminates the influence of scale differences and clearly demonstrates consistent convergence behavior across all stages. The results further validate the robustness and convergence effectiveness of the proposed GNN-PPO framework under increasing problem complexity.

During performance evaluation, the trained model is executed multiple times for each test instance. Actions are sampled from the learned policy distribution, and the best schedule among 10 independent runs is retained to mitigate stochastic variance. Since aircraft assembly scheduling can be regarded as a variant of the Resource-Constrained Project Scheduling Problem (RCPSP), three categories of benchmark algorithms are selected for comprehensive comparison.

First, five high-performance priority-rule-based dispatching heuristics reported in Cai et al. (2024) are adopted as classical baselines, including GRPW (Greatest Resource Priority Weight), FCFS (First Come, First Serve), EDDF (Earliest Due Date First), MINLFT (Minimum Latest Finish Time), and MTS (Minimum Total Slack). These priority-rule-based heuristics represent traditional dispatching strategies widely used in RCPSP environments.

Second, two representative evolutionary algorithms are included: the Improved Genetic Algorithm (IGA) and the Q-learning Improved Genetic Algorithm (QIGA), implemented following Wen et al. (2024). The implementation strictly follows the algorithmic structures and

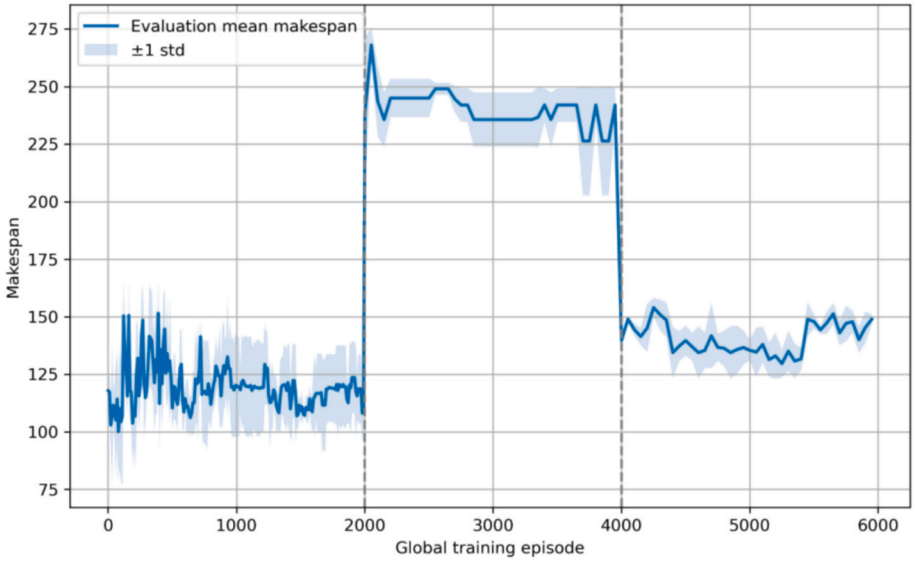


Fig. 10. Convergence curve of evaluation mean makespan during curriculum training.

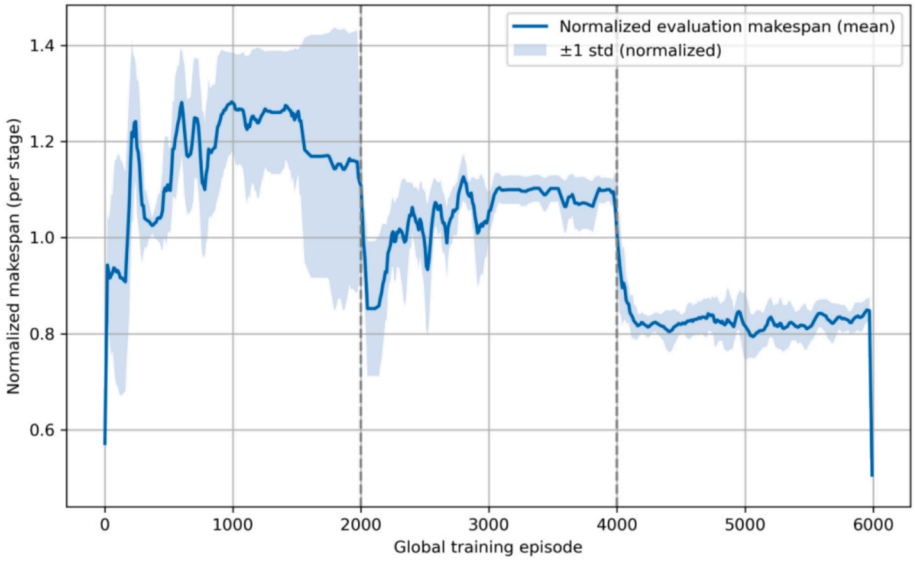


Fig. 11. Normalized convergence curve.

operator settings described in their study to ensure reproducibility and fair comparison.

Third, to further strengthen the comparison with state-of-the-art deep reinforcement learning approaches, a Graph Isomorphism Network-based Double Deep Q Network (GIN-DDQN) is additionally implemented. The algorithmic structure follows Liu et al. (2024), where double Q-learning and multi-step return are incorporated to improve value estimation stability in RCPSP-like environments. To ensure methodological fairness, GIN-DDQN adopts the same graph-based state encoding architecture as the proposed GIN-PPO framework, differing only in the reinforcement learning paradigm (value-based versus policy-gradient-based optimization).

The hyperparameter settings of IGA and QIGA are presented in Table 5, while the detailed configuration of GIN-DDQN is summarized in Table 6. The hyperparameter configuration of GIN-DDQN follows the training mechanism described in Liu et al. (2024), including double Q-learning and multi-step return. The specific parameter values are slightly adjusted to accommodate the scale and constraint characteristics of the aircraft assembly scheduling problem.

The proposed method, after being trained on sizes 30, 50, and 100, is directly applied to test sets of all scales. Moreover, all comparative algorithms are implemented under the proposed GIN framework, utilizing graph isomorphism network encoding, and the results from the comparison algorithms are computed based on this encoding. The detailed comparison results are presented in Table 7.

As shown in Table 7 and Fig. 12, the proposed GIN-PPO algorithm consistently achieves the best or near-best performance across different instance scales when compared with priority-rule-based heuristics, evolutionary algorithms, and the learning-based GIN-DDQN approach. In terms of makespan minimization, GIN-PPO maintains a clear advantage, particularly on medium- and large-scale instances, where the performance gap becomes more pronounced. Compared with traditional priority-rule heuristics, the learning-based approaches exhibit significantly improved solution quality, highlighting the effectiveness of data-driven scheduling policies in complex assembly environments.

Table 5
Parameter Settings for the QIGA.

Popsize	Iterations	P_c	P_m
100	200	0.7	0.08
gamma	epsilon	epsilon_decay	
0.95	0.2	0.99	

Table 6
Parameter Settings for the GIN-DDQN.

Learning rate	γ	Batch size	Memory size	Target update	Multi-step
5e-4	0.99	128	20,000	50	2
ε−start	ε−end	ε−decay_rate	Warmup steps	Hidden units	
0.95	1e-4	1/2000	2000	128	

Furthermore, although GIN-DDQN also benefits from graph-based state encoding, the proposed GIN-PPO framework demonstrates more stable and superior performance, indicating that policy-gradient-based optimization is more suitable for large-scale constrained scheduling problems. These results demonstrate that the proposed method is capable of learning transferable scheduling strategies from small- and medium-scale instances and generalizing effectively to larger-scale problems, thereby validating its robustness and scalability.

To statistically verify the superiority of the proposed GIN-PPO method, paired Wilcoxon signed-rank tests were conducted separately for each problem scale on the paired makespan results of the corresponding test instances at a significance level of $\alpha = 0.05$. Specifically, for each scale (30, 50, 100, 300, and 800 jobs), the Wilcoxon test was performed on the paired makespan values of the 10 test instances within that scale. The test evaluates whether the median performance difference between two paired algorithms is zero without assuming normality. A p-value smaller than 0.05 indicates that the performance improvement of GIN-PPO is statistically significant. The statistical results are summarized in Table 8.

To further quantify the robustness of the proposed approach, the optimal-achievement rate is defined as:

$$Optimal \quad Rate = \frac{N_{best}}{N_{total}} \times 100\% \tag{35}$$

where N_{best} denotes the number of instances in which GIN-PPO achieves the minimum makespan among all compared algorithms (including priority-rule heuristics, evolutionary algorithms, and GIN-DDQN).

In each test instance, the best makespan among 10 independent runs of GIN-PPO is recorded and compared against the final results of all benchmark algorithms under identical settings. Across the 50 test instances, GIN-PPO achieves the minimum makespan in 48 instances, corresponding to an optimal-achievement rate of 96%.

Table 7
Performance Comparison of Different Algorithms.

Size		GRPW	FCFS	EDDF	MINLFT	MTS	IGA	QIGA	GIN-DDQN	GIN-PPO
J30-1	Makespan	100.29	92.12	93.22	92.12	92.12	93.12	98.4	92.17	**88.09**
	Gap	13.85%	4.57%	5.82%	4.57%	4.57%	5.71%	11.7%	4.63%	0
	Time(s)	0.0269	0.01	0.01	0.012	0.01	98.329	130.325	0.04	0.0485
J30-2	Makespan	86.99	75.98	79.94	85.97	77.05	78.14	95.33	79.24	**73.97**
	Gap	17.6%	2.72%	8.07%	16.22%	4.16%	5.64%	28.88%	7.12%	0
	Time(s)	0.0195	0.015	0.013	0.0149	0.012	122.1	155.693	0.0548	0.0478
J30-3	Makespan	108.92	118.21	125.13	110.13	118.21	125.03	144.03	128.65	**102.09**
	Gap	6.69%	15.79%	22.57%	7.88%	15.79%	22.47%	41.08%	26.02%	0
	Time(s)	0.0279	0.014	0.013	0.013	0.0139	161.47	162.99	0.0748	0.0518
J30-4	Makespan	101.48	88.81	84.97	99.64	93.4	93.97	87.08	89.38	**83.59**
	Gap	21.4%	6.24%	1.65%	19.2%	11.74%	12.42%	4.18%	6.93%	0
	Time(s)	0.0259	0.0139	0.014	0.0159	0.0149	125.38	166.562	0.0579	0.0531
J30-5	Makespan	100.83	91.38	100.28	92.95	92.51	99.61	109.88	84.83	**77.81**
	Gap	29.58%	17.44%	28.88%	19.46%	18.89%	28.02%	41.22%	9.02%	0
	Time(s)	0.0289	0.011	0.014	0.0155	0.0149	120.499	111.704	0.0438	0.0458
J30-6	Makespan	92.85	88.8	96.78	95.76	88.8	96.98	103.36	90.93	**81.99**
	Gap	13.25%	8.31%	18.04%	16.79%	8.31%	18.28%	26.06%	10.9%	0
	Time(s)	0.0309	0.013	0.011	0.0159	0.013	161.757	137.558	0.0581	0.0498
J30-7	Makespan	80.93	68.37	76.21	72.72	72.61	69.61	81.19	76.38	**66.11**
	Gap	22.42%	3.42%	15.28%	10.0%	9.83%	5.29%	22.81%	15.53%	0
	Time(s)	0.0279	0.013	0.013	0.016	0.013	225.954	216.404	0.0524	0.0461
J30-8	Makespan	116.03	100.02	109.19	121.19	97.05	105.02	110.41	105.08	**90.16**
	Gap	28.69%	10.94%	21.11%	34.42%	7.64%	16.48%	22.46%	16.55%	0
	Time(s)	0.0369	0.0229	0.0159	0.013	0.012	237.966	329.723	0.0616	0.049
J30-9	Makespan	74.06	74.15	76.67	80.21	77.82	76.67	82.08	74.04	**67.64**
	Gap	9.49%	9.62%	13.35%	18.58%	15.05%	13.35%	21.35%	9.46%	0
	Time(s)	0.0259	0.0179	0.018	0.0236	0.016	294.647	214.311	0.0511	0.0488
J30-10	Makespan	95.04	84.91	81.97	103.06	93.24	81.97	101.14	86.57	**78.98**
	Gap	20.33%	7.51%	3.79%	30.49%	18.06%	3.79%	28.06%	9.61%	0
	Time(s)	0.0459	0.0189	0.0159	0.0279	0.0194	232.327	257.047	0.0677	0.0528
J50-1	Makespan	242.17	233.2	231.13	240.03	241.64	252.01	269.18	246.06	**220.01**
	Gap	10.07%	6.0%	5.05%	9.1%	9.83%	14.54%	22.35%	11.84%	0
	Time(s)	0.0618	0.0618	0.057	0.0644	0.0548	558.11	584.34	0.111	0.131
J50-2	Makespan	171.32	168.76	163.84	160.5	175.85	163.51	171.96	165.99	**153.52**
	Gap	11.59%	9.93%	6.72%	4.55%	14.55%	6.51%	12.01%	8.12%	0
	Time(s)	0.0538	0.0439	0.0398	0.0468	0.0528	508.258	437.97	0.136	0.102
J50-3	Makespan	**146.29**	148.26	152.15	161.68	148.26	148.2	177.5	170.17	149.11
	Gap	0	1.35%	4.01%	10.52%	1.35%	1.31%	21.33%	16.32%	1.93%
	Time(s)	0.0927	0.0349	0.0389	0.0399	0.0355	467.686	515.507	0.109	0.096
J50-4	Makespan	213.36	148.25	166.34	154.11	**139.31**	174.48	187.54	176.43	144.39
	Gap	53.15%	6.42%	19.4%	10.62%	0	25.25%	34.62%	26.65%	3.65%
	Time(s)	0.0578	0.0349	0.0439	0.0419	0.0339	558.103	491.369	0.118	0.095
J50-5	Makespan	225.01	228.09	221.01	221.97	235.0	227.84	225.16	227.99	**206.19**
	Gap	9.13%	10.62%	7.19%	7.65%	13.97%	10.5%	9.2%	10.57%	0
	Time(s)	0.0675	0.0508	0.0488	0.0568	0.0545	413.047	447.133	0.125	0.105
J50-6	Makespan	166.99	140.23	153.32	147.89	139.21	147.21	156.38	154.18	**135.97**
	Gap	22.81%	3.13%	12.76%	8.77%	2.38%	8.27%	15.01%	13.39%	0
	Time(s)	0.0718	0.0379	0.0409	0.0439	0.0429	562.123	599.43	0.133	0.094
J50-7	Makespan	233.23	223.05	211.1	224.21	235.1	228.1	271.56	214.1	**208.02**
	Gap	12.12%	7.23%	1.48%	7.78%	13.02%	9.65%	30.55%	2.92%	0
	Time(s)	0.0638	0.0578	0.0508	0.0698	0.0568	465.45	537.72	0.114	0.102
J50-8	Makespan	221.02	228.09	211.07	214.1	228.09	231.05	245.6	219.07	**198.2**
	Gap	11.51%	15.08%	6.49%	8.02%	15.08%	16.57%	23.92%	10.53%	0
	Time(s)	0.0698	0.0528	0.0468	0.0545	0.0538	542.23	676.48	0.11	0.107
J50-9	Makespan	**146.78**	160.27	161.11	168.1	159.27	161.11	189.48	187.1	154.23
	Gap	0	9.19%	9.76%	14.53%	8.51%	9.76%	29.09%	27.47%	5.08%
	Time(s)	0.0967	0.0399	0.0379	0.0484	0.0399	539.71	638.81	0.135	0.097
J50-10	Makespan	245.16	235.13	222.23	240.07	220.2	228.23	262.12	243.09	**209.12**
	Gap	17.23%	12.44%	6.27%	14.8%	5.3%	9.14%	25.34%	16.24%	0
	Time(s)	0.0708	0.0628	0.0508	0.0624	0.0568	619.56	746.07	0.139	0.109
J100-1	Makespan	345.11	338.19	310.02	322.04	365.1	349.02	340.38	320.06	**309.17**
	Gap	11.62%	9.39%	0.27%	4.16%	18.09%	12.89%	10.09%	3.52%	0
	Time(s)	0.933	0.66	0.543	0.694	0.988	2020.92	1727.64	0.857	0.598
J100-2	Makespan	327.03	321.11	339.72	331.39	329.97	314.11	309.21	312.04	**289.5**
	Gap	12.96%	10.92%	17.35%	14.47%	13.98%	8.5%	6.81%	7.79%	0
	Time(s)	0.555	0.404	0.456	0.411	0.474	1848.47	1773.69	1.108	0.57
J100-3	Makespan	313.37	313.25	291.35	276.53	292.08	289.17	321.9	292.08	**273.45**
	Gap	14.6%	14.55%	6.55%	1.13%	6.81%	5.75%	17.72%	6.81%	0
	Time(s)	0.542	0.466	0.384	0.438	0.486	1652.4	1471.77	1.188	1.472
J100-4	Makespan	341.1	353.86	344.84	336.4	351.74	357.62	364.68	336.08	**321.28**
	Gap	6.17%	10.14%	7.33%	4.71%	9.48%	11.31%	13.51%	4.61%	0
	Time(s)	0.511	0.452	0.444	0.437	0.524	1890.41	1776.64	0.984	1.553
J100-5	Makespan	393.37	382.66	364.57	403.33	388.27	383.59	440.05	368.24	**352.12**

(*continued on next page*)

Table 7 (*continued*)

Size		GRPW	FCFS	EDDF	MINLFT	MTS	IGA	QIGA	GIN-DDQN	GIN-PPO
	Gap	11.71%	8.67%	3.54%	14.54%	10.27%	8.94%	24.97%	4.58%	0
	Time(s)	0.571	0.448	0.433	0.493	0.503	1947.79	2041.72	1.138	1.67
J100-6	Makespan	331.34	334.15	338.12	350.21	374.15	363.62	374.26	328.08	**314.2**
	Gap	5.46%	6.35%	7.61%	11.46%	19.08%	15.73%	19.12%	4.42%	0
	Time(s)	0.524	0.429	0.429	0.422	0.547	1916.7	2180.78	1.057	1.693
J100-7	Makespan	330.39	318.27	323.48	343.56	365.22	356.57	354.22	327.08	**303.14**
	Gap	8.99%	4.99%	6.71%	13.33%	20.48%	17.63%	16.85%	7.9%	0
	Time(s)	0.525	0.394	0.414	0.46	0.558	2050.74	1404.87	1.075	1.634
J100-8	Makespan	343.12	366.16	363.1	344.86	343.71	372.5	375.42	343.02	**298.16**
	Gap	15.08%	22.81%	21.78%	15.66%	15.28%	24.93%	25.91%	15.05%	0
	Time(s)	0.535	0.459	0.464	0.449	0.467	1939.75	2111.43	0.826	1.708
J100-9	Makespan	386.43	370.12	386.29	389.07	362.09	392.29	417.57	381.78	**354.65**
	Gap	8.96%	4.36%	8.92%	9.71%	2.1%	10.61%	17.74%	7.65%	0
	Time(s)	0.495	0.385	0.391	0.418	0.443	1640.13	1349.06	0.951	0.991
J100-10	Makespan	390.27	366.46	374.86	371.23	357.06	411.86	393.89	381.28	**341.05**
	Gap	14.43%	7.45%	9.91%	8.85%	4.69%	20.76%	15.49%	11.8%	0
	Time(s)	0.616	0.441	0.451	0.488	0.509	1990.1	2088.66	2.34	1.754
J300-1	Makespan	313.82	320.14	315.89	306.84	311.68	307.37	312.77	312.85	**302.66**
	Gap	3.69%	5.78%	4.37%	1.38%	2.98%	1.56%	3.34%	3.37%	0
	Time(s)	11.792	11.353	11.207	11.319	11.271	3083.11	5797.27	9.3	12.5
J300-2	Makespan	237.28	242.51	238.13	233.07	247.3	243.11	254.99	237.31	**214.68**
	Gap	10.53%	12.96%	10.92%	8.57%	15.19%	13.24%	18.78%	10.54%	0
	Time(s)	19.941	19.543	19.531	19.524	16.219	3584.3	6179.03	19.08	9.547
J300-3	Makespan	610.6	624.04	611.18	604.17	627.66	619.56	643.5	622.5	**593.1**
	Gap	2.95%	5.22%	3.05%	1.87%	5.83%	4.46%	8.5%	4.96%	0
	Time(s)	11.843	11.593	11.638	11.854	11.853	3281.79	6829.16	9.89	9.469
J300-4	Makespan	587.37	584.66	580.8	577.62	609.4	595.53	614.54	575.58	**569.2**
	Gap	3.19%	2.72%	2.04%	1.48%	7.06%	4.63%	7.97%	1.12%	0
	Time(s)	12.345	12.108	12.104	12.172	12.977	2987.3	5394.3	10.05	19.02
J300-5	Makespan	539.04	555.2	546.14	535.15	567.91	558.44	556.72	545.01	**519.8**
	Gap	3.7%	6.81%	5.07%	2.95%	9.26%	7.43%	7.1%	4.85%	0
	Time(s)	12.257	11.841	11.854	11.812	11.869	3082.75	4864.24	9.91	17.84
J300-6	Makespan	538.55	536.68	524.96	522.43	537.98	527.16	523.1	530.16	**506.2**
	Gap	6.39%	6.02%	3.71%	3.21%	6.28%	4.14%	3.34%	4.73%	0
	Time(s)	12.419	12.192	12.164	12.144	12.331	3385.14	5277.06	10.03	22.84
J300-7	Makespan	560.87	565.02	561.49	551.46	582.35	545.01	555.14	551.37	**539.3**
	Gap	4.0%	4.77%	4.11%	2.25%	7.98%	1.06%	2.94%	2.24%	0
	Time(s)	12.385	11.962	11.919	11.932	12.086	3383.62	4987.56	10.29	25.23
J300-8	Makespan	512.77	525.82	537.36	518.9	544.36	518.41	545.97	520.72	**507.09**
	Gap	1.12%	3.69%	5.97%	2.33%	7.35%	2.23%	7.67%	2.69%	0
	Time(s)	12.034	11.761	11.828	11.775	11.828	3182.73	4981.48	9.77	25.46
J300-9	Makespan	595.64	626.47	604.69	596.88	618.05	591.17	627.54	606.66	**587.38**
	Gap	1.41%	6.65%	2.95%	1.62%	5.22%	0.65%	6.84%	3.28%	0
	Time(s)	12.587	12.466	12.432	12.459	12.649	3687.2	5377.03	10.534	18.47
J300-10	Makespan	619.6	622.19	626.95	615.13	634.92	631.7	619.57	624.12	**607.27**
	Gap	2.03%	2.46%	3.24%	1.29%	4.55%	4.02%	2.03%	2.77%	0
	Time(s)	12.487	12.332	18.944	20.786	21.01	3288.59	5322.3	18.67	9.907
J800-1	Makespan	1143.98	1110.88	1103.29	1089.22	1139.59	1090.48	1114.01	1136.19	**1061.66**
	Gap	7.75%	4.64%	3.92%	2.6%	7.34%	2.71%	4.93%	7.02%	0
	Time(s)	74.583	63.419	62.938	63.929	64.662	6468.85	21596.63	62.61	59.55
J800-2	Makespan	1133.64	1082.02	1046.06	1089.93	1147.86	1095.97	1165.04	1138.23	**1017.15**
	Gap	11.45%	6.38%	2.84%	7.16%	12.85%	7.75%	14.54%	11.9%	0
	Time(s)	128.454	113.486	112.134	113.076	118.261	6461.2	22566.21	62.9	60.36
J800-3	Makespan	1073.09	1019.84	1023.44	1024.3	1108.66	1050.65	1104.68	1099.12	**1004.35**
	Gap	6.84%	1.54%	1.9%	1.99%	10.39%	4.61%	9.99%	9.44%	0
	Time(s)	128.811	112.752	113.665	113.907	116.474	6457.29	20754.6	59.86	59.76
J800-4	Makespan	1121.25	1030.8	1049.3	1064.72	1113.43	1055.52	1092.65	1113.3	**1024.97**
	Gap	9.39%	0.57%	2.37%	3.88%	8.63%	2.98%	6.6%	8.62%	0
	Time(s)	81.428	62.776	63.154	63.892	64.362	6460.3	22573.65	61.43	51.53
J800-5	Makespan	1215.53	1141.84	1127.64	1129.79	1186.44	1147.99	1135.81	1195.49	**1111.41**
	Gap	9.37%	2.74%	1.46%	1.65%	6.75%	3.29%	2.2%	7.57%	0
	Time(s)	73.8	63.54	62.468	62.501	63.857	6462.49	22396.91	63.49	134.86
J800-6	Makespan	1102.05	1069.05	1040.13	1041.36	1122.9	1084.05	1117.8	1129.42	**1013.96**
	Gap	8.69%	5.43%	2.58%	2.7%	10.74%	6.91%	10.24%	11.39%	0
	Time(s)	71.725	63.223	62.755	63.009	64.177	6459.1	21959.81	61.92	139.23
J800-7	Makespan	1195.92	1094.25	1109.66	1129.49	1180.34	1124.29	1142.93	1162.5	**1069.77**
	Gap	11.79%	2.29%	3.73%	5.58%	10.34%	5.1%	6.84%	8.67%	0
	Time(s)	73.517	63.351	62.69	62.997	64.333	6460.5	23497.21	63.76	136.12
J800-8	Makespan	1107.74	1056.82	1073.06	1095.34	1149.93	1077.72	1153.46	1120.3	**1049.43**
	Gap	5.56%	0.7%	2.25%	4.37%	9.58%	2.7%	9.91%	6.75%	0
	Time(s)	72.416	62.89	63.614	62.946	65.576	6461.69	21393.69	64.38	112.91
J800-9	Makespan	1076.21	1066.56	1027.93	1021.29	1104	1065.67	1098.9	1096.85	**1014.21**
	Gap	6.11%	5.16%	1.35%	0.7%	8.85%	5.07%	8.35%	8.15%	0
	Time(s)	72.67	62.583	61.358	61.542	63.474	6460.02	21492.09	63.29	165.91
J800-10	Makespan	1188.76	1137.82	1133.96	1151.63	1196.88	1140.12	1166.8	1192.63	**1118.29**

(*continued on next page*)

Table 7 (*continued*)

Size		GRPW	FCFS	EDDF	MINLFT	MTS	IGA	QIGA	GIN-DDQN	GIN-PPO
	Gap	6.3%	1.75%	1.4%	2.98%	7.03%	1.95%	4.34%	6.65%	0
	Time(s)	74.667	64.256	64.221	64.279	98.279	6465.41	22298.31	64.93	61.63

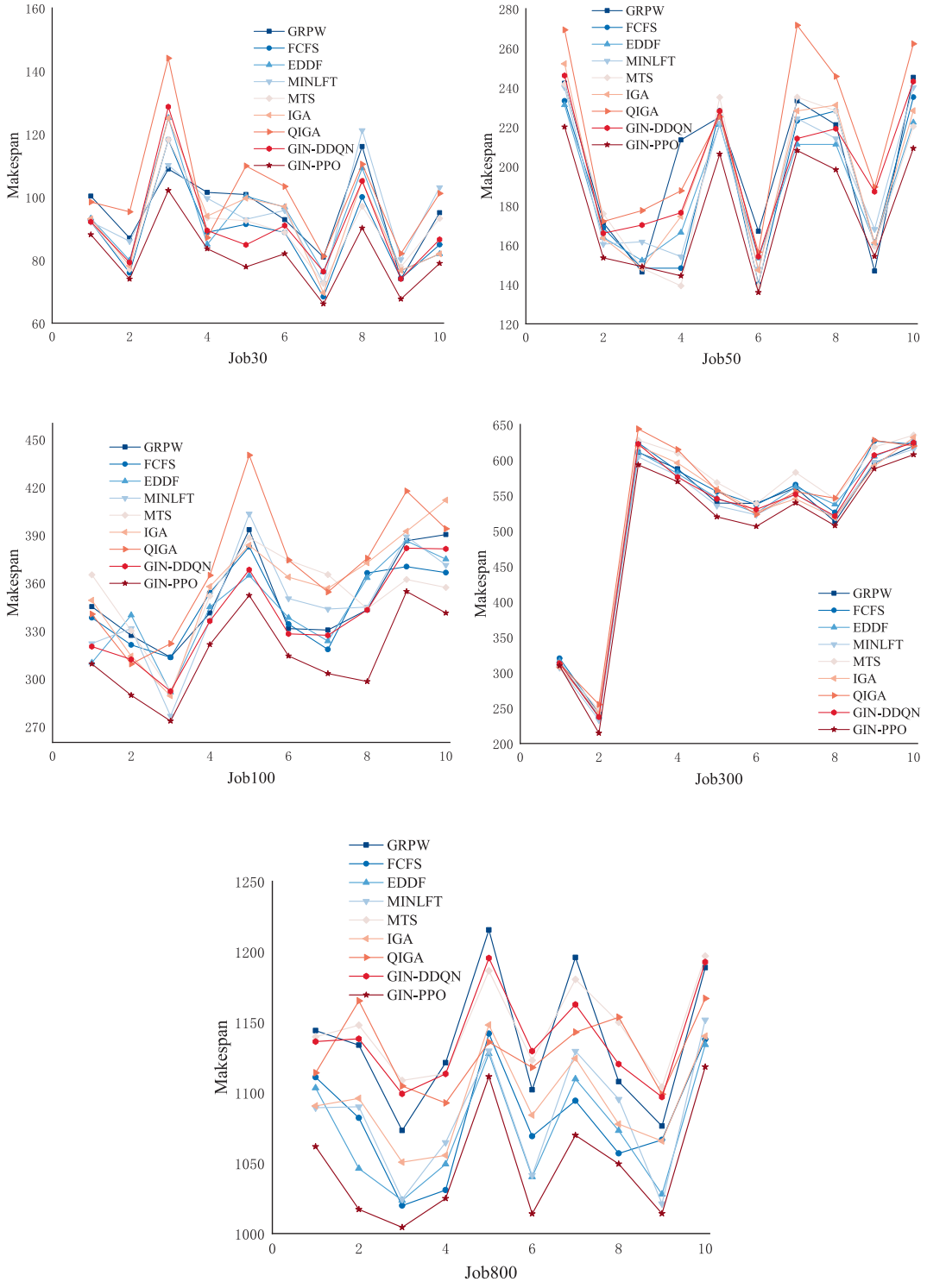


Fig. 12. Scatter Plot Comparing the Performance of Different Algorithms.

Table 8
Results of paired Wilcoxon signed-rank tests on makespan across different instance scales.

Compared algorithms	30	50	100	300	800
(GIN-PPO,GRPW)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,FCFS)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,EDDF)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,MINLFT)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,MTS)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,IGA)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,QIGA)	<0.001	<0.001	<0.001	<0.001	<0.001
(GIN-PPO,GIN-DDQN)	<0.01	<0.05	<0.01	<0.01	<0.01

6. Discussion

The experimental results presented in Table 7 and Fig. 12 demonstrate that the proposed GIN-PPO algorithm achieves consistently superior performance across aircraft assembly scheduling instances of varying scales. Overall, GIN-PPO attains the best makespan in 48 out of 50 test instances, indicating strong robustness and scalability. Compared with traditional priority-rule-based heuristics, the proposed method shows better adaptability to complex precedence and resource constraints. Compared with evolutionary algorithms such as IGA and QIGA, it achieves a better balance between solution quality and computational efficiency.

In terms of computational efficiency, GIN-PPO demonstrates favorable scalability. The average runtime increases from approximately 0.05 s for J30 instances to roughly 60–165 s for J800 instances, reflecting polynomial growth with respect to problem size. By contrast, QIGA requires approximately 21,596 s for instance J800-1, and nearly 4,987 s for J300-7, representing about a 200-fold increase in computational time compared with GIN-PPO. While priority rules are computationally inexpensive, their inferior solution quality limits their applicability in large-scale industrial scenarios. GIN-PPO achieves a balanced trade-off between solution optimality and computational efficiency, making it suitable for practical deployment.

A direct comparison between GIN-PPO and GIN-DDQN further highlights the effectiveness of the proposed framework. Although both approaches adopt the same GIN-based graph representation, GIN-PPO consistently outperforms GIN-DDQN in terms of makespan across most instance scales. This difference can be attributed to the underlying reinforcement learning paradigms. GIN-DDQN relies on value-based learning with max-operator-based action selection, which may suffer from overestimation bias and instability in large, dynamically constrained action spaces. In contrast, PPO directly optimizes a stochastic policy through a clipped surrogate objective, enabling more stable updates and better adaptation to varying feasible action sets in AASP environments. The results suggest that the integration of GIN representation learning with policy-gradient-based optimization forms a more compatible and effective framework for large-scale constrained scheduling problems.

Overall, the findings confirm that the proposed GIN-PPO approach not only delivers superior optimization performance but also maintains computational feasibility and strong generalization capability across different problem scales, demonstrating its potential for real-world aircraft assembly scheduling applications.

7. Conclusion

1. This paper addresses the challenges of skill-level constraints and multi-resource coordination in aircraft pulsating assembly scheduling by proposing an intelligent scheduling framework that integrates Graph Isomorphism Networks (GIN) with Proximal Policy Optimization (PPO). The proposed method effectively models complex precedence relationships and heterogeneous resource interactions within large-scale assembly systems.
2. By combining graph-based representation learning with stable policy-gradient optimization, the GIN-PPO framework overcomes the limitations of traditional heuristic and evolutionary approaches, which often struggle to balance solution quality and computational efficiency in large-scale constrained scheduling problems.
3. Experimental results demonstrate that GIN-PPO achieves the best makespan in 48 out of 50 test instances (96%) across problem sizes ranging from 30 to 800. Compared with evolutionary algorithms such as QIGA, the proposed method reduces computational time by up to 200 times while maintaining superior solution quality. Furthermore, the model exhibits strong scalability and generalization capability, consistently maintaining low performance gaps as problem complexity increases.
4. The results also indicate that policy-gradient-based optimization is more suitable than value-based learning (e.g., GIN-DDQN) for handling dynamically constrained combinatorial scheduling environments, highlighting the effectiveness of integrating GIN representation with PPO optimization.
5. Future work will focus on incorporating transfer learning and attention mechanisms to further enhance cross-scale generalization and improve representation efficiency for larger graphs. Additionally, extending the framework to dynamic and disturbance-driven scheduling scenarios will further strengthen its applicability in intelligent manufacturing systems.

CRediT authorship contribution statement

Xiaoyu Wen: Writing – original draft, Validation, Methodology, Formal analysis. **Hao Zhang:** Writing – review & editing, Investigation, Data curation. **Wenchao Yang:** Validation, Data curation. **Hao Li:** Writing – review & editing, Supervision. **Haoqi Wang:** Investigation, Formal analysis. **Hongwen Xing:** Validation, Resources. **Siren Liu:** Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research work is supported by the National Natural Science Foundation of China (Grant no.52475543),Natural Science Foundation of Henan (Grant No.252300421101); Henan Province University Science and Technology Innovation Talent Support Plan (Grant no.24HASTIT048), Science and Technology Innovation Team Project of Zhengzhou University of Light Industry (Grant no.23XNKJTD010).

Thank the “Central Plains Science and Technology Innovation Youth Top Talent” Program of the Science and Technology Association of Henan Province for their generous support.

Data availability

Data will be made available on request.

References

Alcaraz, J., Maroto, C., & Ruiz, R. (2003). Solving the Multi-mode resource-constrained project scheduling problem with genetic algorithms. *Journal of the Operational Research Society, 54*(6), 614–626. https://doi.org/10.1057/palgrave.jors.2601563

Bahroun, Z., As’ ad, R., Tanash, & Athamneh, R. (2024). The multi-skilled resource-constrained project scheduling problem: systematic review and an exploration of future landscapes. management systems. *Production Engineering, 1 (32*, 108–132. https://doi.org/10.2478/mspe-2024-0012

Bao, Z., Chen, L., & Qiu, K. (2023a). A surrogate-assisted heuristic approach for the joint optimization of resource allocation and scheduling of an aircraft final assembly line.

Journal of Manufacturing Systems, 70, 99–112. https://doi.org/10.1016/j.jmsy.2023.07.003

Bao, Z., Chen, L., & Qiu, K. (2023b). An aircraft final assembly line balancing problem considering resource constraints and parallel task scheduling. *Computers & Industrial Engineering, 182*, Article 109436. https://doi.org/10.1016/j.cie.2023.109436

Bao, Z., & Chen, L. (2025). Ramp-up planning for aircraft assembly production considering learning effect. Computers & Industrial Engineering, 111336 10.1016/j.cie.2025.111336.

Blanco, D., Rubio, E. M., Agustina, B., Marín, M. M., & Camacho, A. M. (2024). Advanced procedures and scheduling for aircraft assembly processes: A systematic review approach. *Procedia CIRP, 126*, 817–822. https://doi.org/10.1016/j.procir.2024.08.264

Blazewicz, J., Dror, M., & Weglarz, J. (1991). Mathematical programming formulations for machine scheduling: A survey. *European Journal of Operational Research, 51*(3), 283–300. https://doi.org/10.1016/0377-2217(91)90304-E

Borreguero Sanchidrián, T., Portoleau, T., Artigues, C., García Sánchez, A., Ortega Mier, M., & Lopez, P. (2024). Large neighborhood search for an aeronautical assembly line time-constrained scheduling problem with multiple modes and a resource leveling objective. *Ann. Oper. Res., 338*(1), 13–40. https://doi.org/10.1007/s10479-023-05629-3

Boudjelida, A. (2019). On the robustness of joint production and maintenance scheduling in presence of uncertainties. *Journal of Intelligent Manufacturing, 30*(4), 1515–1530. https://doi.org/10.1007/s10845-017-1303-9

Brucker, P., & Knust, S. (2000). A linear programming and constraint propagation-based lower bound for the RCPSP. *European Journal of Operational Research, 127*(2), 355–362. https://doi.org/10.1016/S0377-2217(99)00489-0

Cai, H., Bian, Y., & Liu, L. (2024). Deep reinforcement learning for solving resource constrained project scheduling problems with resource disruptions. *Robotics and Computer-Integrated Manufacturing, 85*, Article 102628. https://doi.org/10.1016/j.rcim.2023.102628

Deblaere, F., Demeulemeester, E., & Herroelen, W. (2011). Reactive scheduling in the multi-mode RCPSP. *Computers & Operations Research, 38*(1), 63–74. https://doi.org/10.1016/j.cor.2010.01.001

Fang, P., Yang, J., Liao, Q., Zhong, R. Y., & Jiang, Y. (2021). Flexible worker allocation in aircraft final assembly line using multiobjective evolutionary algorithms. *IEEE Transactions on Industrial Informatics, 17*(11), 7468–7478. https://doi.org/10.1109/TII.2021.3051896

Guo, D. (2024). Fast scheduling of human-robot teams collaboration on synchronised production-logistics tasks in aircraft assembly. *Robotics and Computer-Integrated Manufacturing, 85*, Article 102620. https://doi.org/10.1016/j.rcim.2023.102620

Hu, J. H., Zhu, J. Y., Sun, Y. N., Chen, Y., & Qin, W. (2024). Network-based two-stage robust scheduling strategy for the aircraft assembly system with uncertain duration. *Computers & Industrial Engineering, 188*, Article 109911. https://doi.org/10.1016/j.cie.2024.109911

Hu, Y., Zhang, L., Wang, Q., Zhang, Z., & Tang, Q. (2024). A matheuristic-based multi-objective evolutionary algorithm for flexible assembly jobs shop scheduling problem in cellular manufacture. *Swarm and Evolutionary Computation, 87*, Article 101549. https://doi.org/10.1016/j.swevo.2024.101549

Huang, Y. Y., Pan, Q. K., Gao, L., Miao, Z. H., & Peng, C. (2022). A two-phase evolutionary algorithm for multi-objective distributed assembly permutation flowshop scheduling problem. *Swarm and Evolutionary Computation, 74*, Article 101128. https://doi.org/10.1016/j.swevo.2022.101128

Kolisch, R., Hartmann, S. (1999). Heuristic Algorithms for the Resource-Constrained Project Scheduling Problem: Classification and Computational Analysis. In: Węglarz, J. (eds) Project Scheduling. International Series in Operations Research & Management Science, vol 14. Springer, Boston, MA. https://doi.org/10.1007/978-1-4615-5533-9_7.

Kolisch, R., & Hartmann, S. (2006). Experimental investigation of heuristics for resource-constrained project scheduling: An update. *European journal of operational research, 174*(1), 23–37. https://doi.org/10.1016/j.ejor.2005.01.065

Li, F., & Xu, Z. (2018). A multi-agent system for distributed multi-project scheduling with two-stage decomposition. *PLoS One1, 13*(10), Article e0205445. https://doi.org/10.1371/journal.pone.0205445

Liu, H., Zhang, J., Zhang, X., et al. (2024). A new resource-constrained project scheduling problem with ladder-type carbon trading prices and its algorithm based on deep reinforcement learning. *Expert Systems with Applications, 255*, Article 124794. https://doi.org/10.1016/j.eswa.2024.124794

Li, T., Hua, M., Wang, J., & Qin, W. (2025). Static scheduling method for aircraft flat-tail assembly production based on improved bi-level genetic algorithm. *Scientific Reports, 15*(1), 9642. https://doi.org/10.1038/s41598-025-94027-9

Lovato, D., Guillaume, R., Thierry, C., & Battaia, O. (2022). Managing disruptions in aircraft assembly lines with staircase criteria. *International Journal of Production Research, 61*(2), 632–648. https://doi.org/10.1080/00207543.2021.2011982

Meng, W., Zheng, Q., Pan, G., & Yin, Y. (2023, June). Off-policy proximal policy optimization. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 37, No. 8, pp. 9162-9170) 10.1609/aaai.v37i8.26099.

Mihoubi, B., Bouzouia, B., & Gaham, M. (2021). Reactive scheduling approach for solving a realistic flexible job shop scheduling problem. *International journal of production research, 59*(19), 5790–5808. https://doi.org/10.1080/00207543.2020.1790686

Mingozzi, A., Maniezzo, V., Ricciardelli, S., & Bianco, L. (1998). An exact algorithm for the resource-constrained project scheduling problem based on a new mathematical formulation. *Management science, 44*(5), 714–729. https://doi.org/10.1287/mnsc.44.5.714

Morandini, S., Fraboni, F., Hall, M., Quintana-Amate, S., & Pietrantoni, L. (2025). Human factors and emerging needs in aerospace manufacturing planning and scheduling. *Cognition, Technology & Work, 27*(1), 59–77. https://doi.org/10.1007/s10111-024-00785-3

Nudtasomboon, N., & Randhawa, S. U. (1997). Resource-constrained project scheduling with renewable and non-renewable resources and time-resource tradeoffs. *Computers & Industrial Engineering, 32*(1), 227–242. https://doi.org/10.1016/S0360-8352(96)00212-4

Olaguíbel, R. A. V., & Goerlich, J. M. T. (1989). Heuristic algorithms for resource-constrained project scheduling: A review and an empirical analysis. *Advances in Project Scheduling*, 113–134.

Pritsker, A. A. B., Waiters, L. J., & Wolfe, P. M. (1969). Multiproject scheduling with limited resources: A zero-one programming approach. *Management Science, 16*(1), 93–108. https://doi.org/10.1287/mnsc.16.1.93

Qiu, J., Liu, J., Li, Z., & Lai, X. (2024). A multi-level action coupling reinforcement learning approach for online two-stage flexible assembly flow shop scheduling. *Journal of Manufacturing Systems, 76*, 351–370. https://doi.org/10.1016/j.jmsy.2024.08.006

Ren, Y., Lu, Z., & Liu, X. (2020). A branch-and-bound embedded genetic algorithm for resource-constrained project scheduling problem with resource transfer time of aircraft moving assembly line. *Optimization Letters, 14*(8), 2161–2195. https://doi.org/10.1007/s11590-020-01542-x

Shan, S., Hu, Z., Liu, Z., Shi, J., Wang, L., & Bi, Z. (2017). An adaptive genetic algorithm for demand-driven and resource-constrained project scheduling in aircraft assembly. *Information Technology and Management, 18*(1), 41–53. https://doi.org/10.1007/s10799-015-0223-7

Sprecher, A. (2000). Scheduling resource-constrained projects competitively at modest memory requirements. *Management Science, 46*(5), 710–723. https://doi.org/10.1287/mnsc.46.5.710.12044

Tereshchuk, V., Stewart, J., Bykov, N., Pedigo, S., Devasia, S., & Banerjee, A. G. (2019). An efficient scheduling algorithm for multi-robot task allocation in assembling aircraft structures. *IEEE Robotics and Automation Letters, 4*(4), 3844–3851. https://doi.org/10.1109/LRA.2019.2929983

Tereshchuk, V., Bykov, N., Pedigo, S., Devasia, S., & Banerjee, A. G. (2021). A scheduling method for multi-robot assembly of aircraft structures with soft task precedence constraints. *Robotics and Computer-Integrated Manufacturing, 71*, Article 102154. https://doi.org/10.1016/j.rcim.2021.102154

Urgo, M., Buergin, J., Tolio, T., & Lanza, G. (2018). Order allocation and sequencing with variable degree of uncertainty in aircraft manufacturing. *CIRP Annals, 67*(1), 431–436. https://doi.org/10.1016/j.cirp.2018.04.038

Villafranca, M., Delgado, F., & Klapp, M. (2025). Aircraft maintenance scheduling under uncertain task processing time. *Transportation Research Part E: Logistics and Transportation Review, 196*, Article 104012. https://doi.org/10.1016/j.tre.2025.104012

Wang, D., Qiao, F., Guan, L., Liu, J., Ding, C., & Shi, J. (2024). Human–machine collaborative optimization method for dynamic worker allocation in aircraft final assembly lines. *Computers & Industrial Engineering, 194*, Article 110370. https://doi.org/10.1016/j.cie.2024.110370

Wang, H., Sarker, B. R., Li, J., & Li, J. (2021). Adaptive scheduling for assembly job shop with uncertain assembly times based on dual Q-learning. *International Journal of Production Research, 59*(19), 5867–5883. https://doi.org/10.1080/00207543.2020.1794075

Wang, H., Lin, W., Peng, T., Xiao, Q., & Tang, R. (2025). Multi-agent deep reinforcement learning-based approach for dynamic flexible assembly job shop scheduling with uncertain processing and transport times. *Expert Systems with Applications, 270*, Article 126441. https://doi.org/10.1016/j.eswa.2025.126441

Wen, X., Zhang, X., Xing, H., Ye, G., Li, H., Zhang, Y., & Wang, H. (2024). An improved genetic algorithm based on reinforcement learning for aircraft assembly scheduling problem. *Computers & Industrial Engineering, 193*, Article 110263. https://doi.org/10.1016/j.cie.2024.110263

Wen, X., Zhang, X., Li, H., Ji, S., Wang, H., Ye, G., & Liu, S. (2025). An improved NSGA-II algorithm based on reinforcement learning for aircraft moving assembly line integration optimization problem. *Swarm and Evolutionary Computation, 94*, Article 101911. https://doi.org/10.1016/j.swevo.2025.101911

Yan, Q., & Wang, H. (2022). Double-layer Q-learning-based joint decision-making of dual resource-constrained aircraft assembly scheduling and flexible preventive maintenance. *IEEE Transactions on aerospace and electronic systems, 58*(6), 4938–4952. https://doi.org/10.1109/TAES.2022.3211247

Yan, Q., Wang, H., & Wu, F. (2022). Digital twin-enabled dynamic scheduling with preventive maintenance using a double-layer Q-learning algorithm. *Computers & Operations Research, 144*, Article 105823. https://doi.org/10.1016/j.cor.2022.105823

Zarei, F., Arashpour, M., Mirnezami, S. A., Shahabi-Shahamiri, R., & Ghasemi, M. (2023). Multi-skill resource-constrained project scheduling problem considering overlapping: Fuzzy multi-objective programming approach to a case study. *International Journal of Construction Management, 24*(8), 820–833. https://doi.org/10.1080/15623599.2023.2260696

Zhang, K., Zhang, Y., Gao, H., & Bi, Z. (2024). Elite gene transfer learning heuristic algorithm in scheduling aircraft assembly. *IEEE Transactions on Industrial Informatics*. https://doi.org/10.1109/TII.2024.3435539

Zhang, W., Du, W., & Du, W. (2025). A knowledge transfer-based co-evolutionary algorithm for dynamic large-scale crude oil scheduling. *Swarm and Evolutionary Computation, 97*, Article 102024. https://doi.org/10.1016/j.swevo.2025.102024

Zheng, J., Zhao, Y., Li, Y., Li, J., Wang, L., & Yuan, D. (2025). Dynamic flexible flow shop scheduling via cross-attention networks and multi-agent reinforcement learning. *Journal of Manufacturing Systems, 80*, 395–411. https://doi.org/10.1016/j.jmsy.2025.03.005

Zhu, H., Lu, Z., Lu, C., & Ren, Y. (2021). A reactive scheduling method for disturbances in aircraft moving assembly line. *International Journal of Production Research, 59*(15), 4756–4772. https://doi.org/10.1080/00207543.2020.1771456